

Lab Sheet 06

Title:

- Programming ESP32 to Interface Input & Output Devices using Wokwi

Aims:

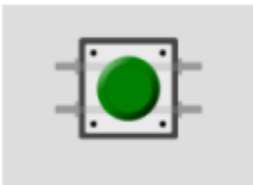
- To learn how to program the ESP32 to correctly read data from input devices and control output devices using the Wokwi online simulator, while understanding the purpose and behavior of each device.

Tasks:

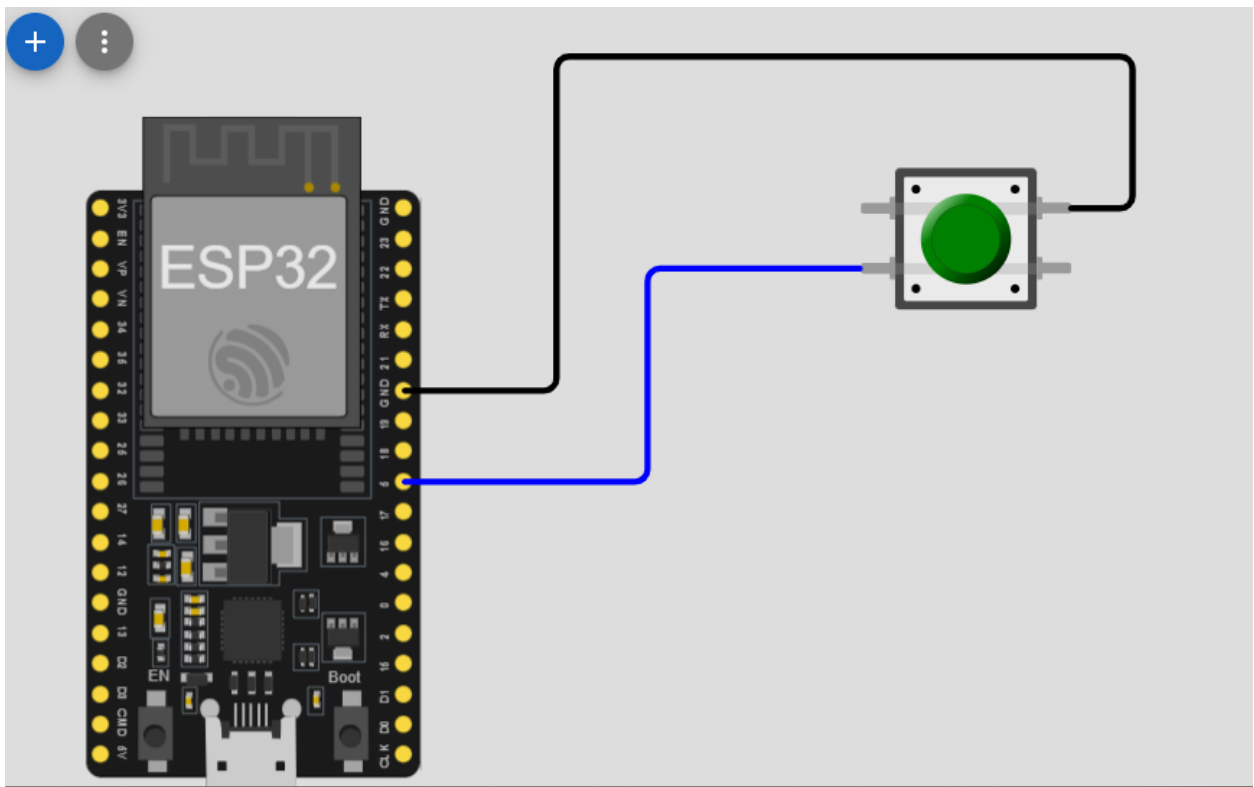
Input Devices

01.Push Button (Tactile Switch)

- Write an ESP32 program to:
 - Detect button press
 - Print “Button Pressed” and “Button Released” on Serial Monitor.



Pins of device	Pins of ESP32
One leg (1.r)	GND
Other leg (2.r)	GPIO 5



```

#include <Arduino.h>
const int BUTTON_PIN = 5; // You can use GPIO 4, 5, 18, etc.

// Variable to store the last known state of the button
int lastButtonState = HIGH;

void setup() {
  // Initialize serial communication at 115200 baud
  Serial.begin(115200);
  Serial.println("Push Button Test");

  // Configure the button pin as an INPUT with internal pull-up resistor
  // This keeps the pin HIGH when the button is NOT pressed.
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  // Read the current state of the button
  int currentButtonState = digitalRead(BUTTON_PIN);

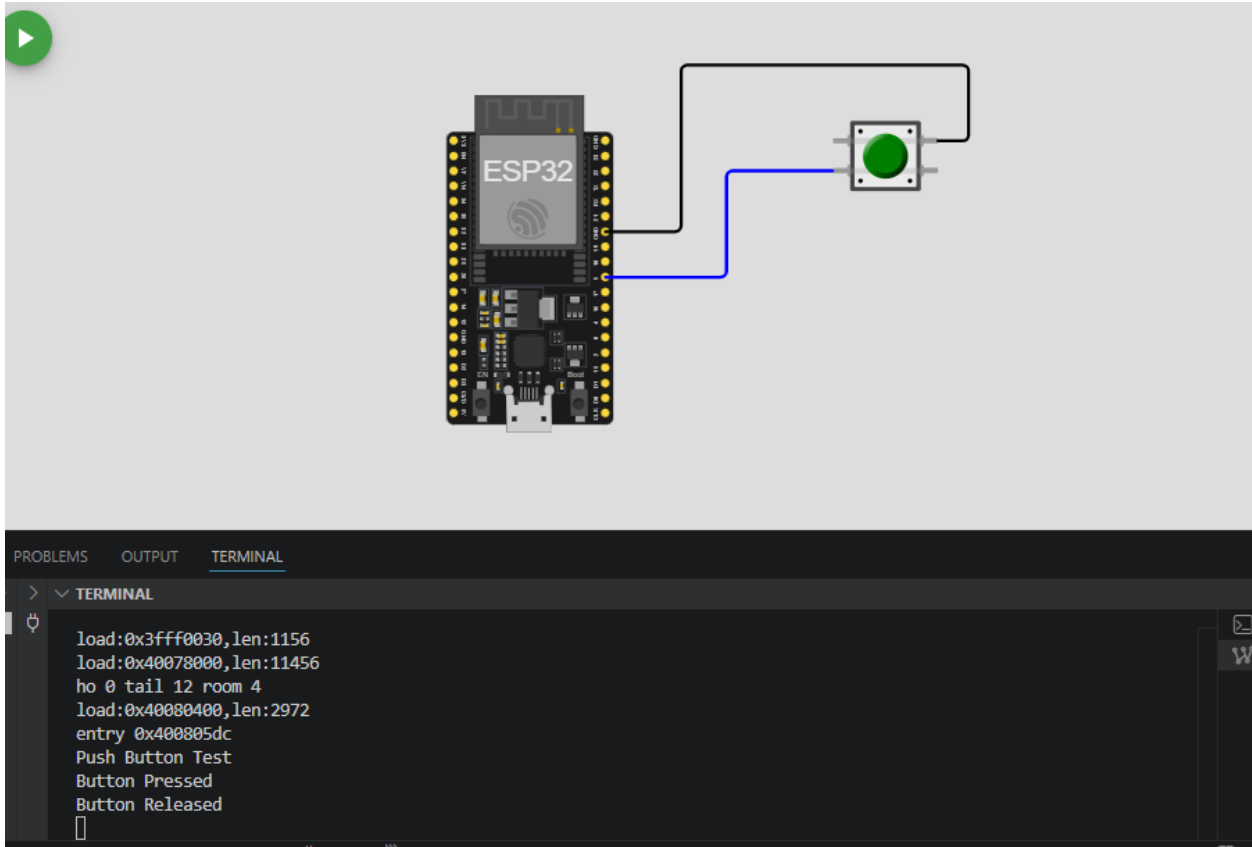
  // Check if the state has changed
  if (currentButtonState != lastButtonState) {

    // If the state went from HIGH to LOW, the button was pressed
    if (currentButtonState == LOW) {
      Serial.println("Button Pressed");
    }
    // If the state went from LOW to HIGH, the button was released
    else {
      Serial.println("Button Released");
    }

    // Save the current state for the next loop iteration
    lastButtonState = currentButtonState;

    // Small delay to debounce (prevent mechanical flicker)
    delay(50);
  }
}

```



02.Slide Switch / Toggle Switch

- Write a program to:
 - Read the switch state
 - Display ON or OFF in Serial Monitor



Pins of device	Pins of ESP32
Middle Pin (2)	GPIO 5
One outer Pin (1)	GND
Other outer pin (3)	-

```
#include <Arduino.h>
const int SWITCH_PIN = 5; // You can use GPIO 4, 5, 18, etc.

// Variable to store the last known state of the switch
int lastSwitchState = -1;

void setup() {
```

```
// Initialize serial communication at 115200 baud
Serial.begin(115200);

Serial.println("Slide Switch / Toggle Switch  Test");

// Configure the switch pin as an INPUT with internal pull-up resistor
pinMode(SWITCH_PIN, INPUT_PULLUP);
}

void loop() {
  // Read the current state of the switch
  int currentSwitchState = digitalRead(SWITCH_PIN);

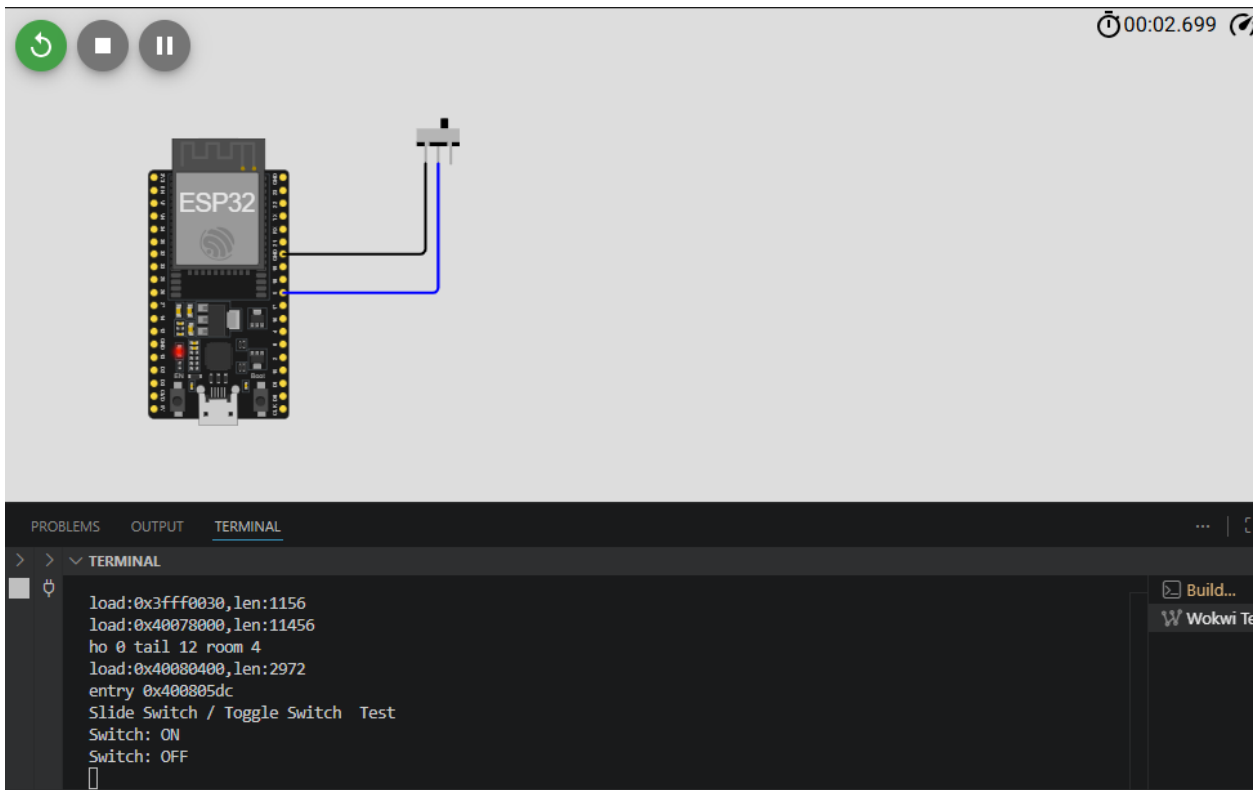
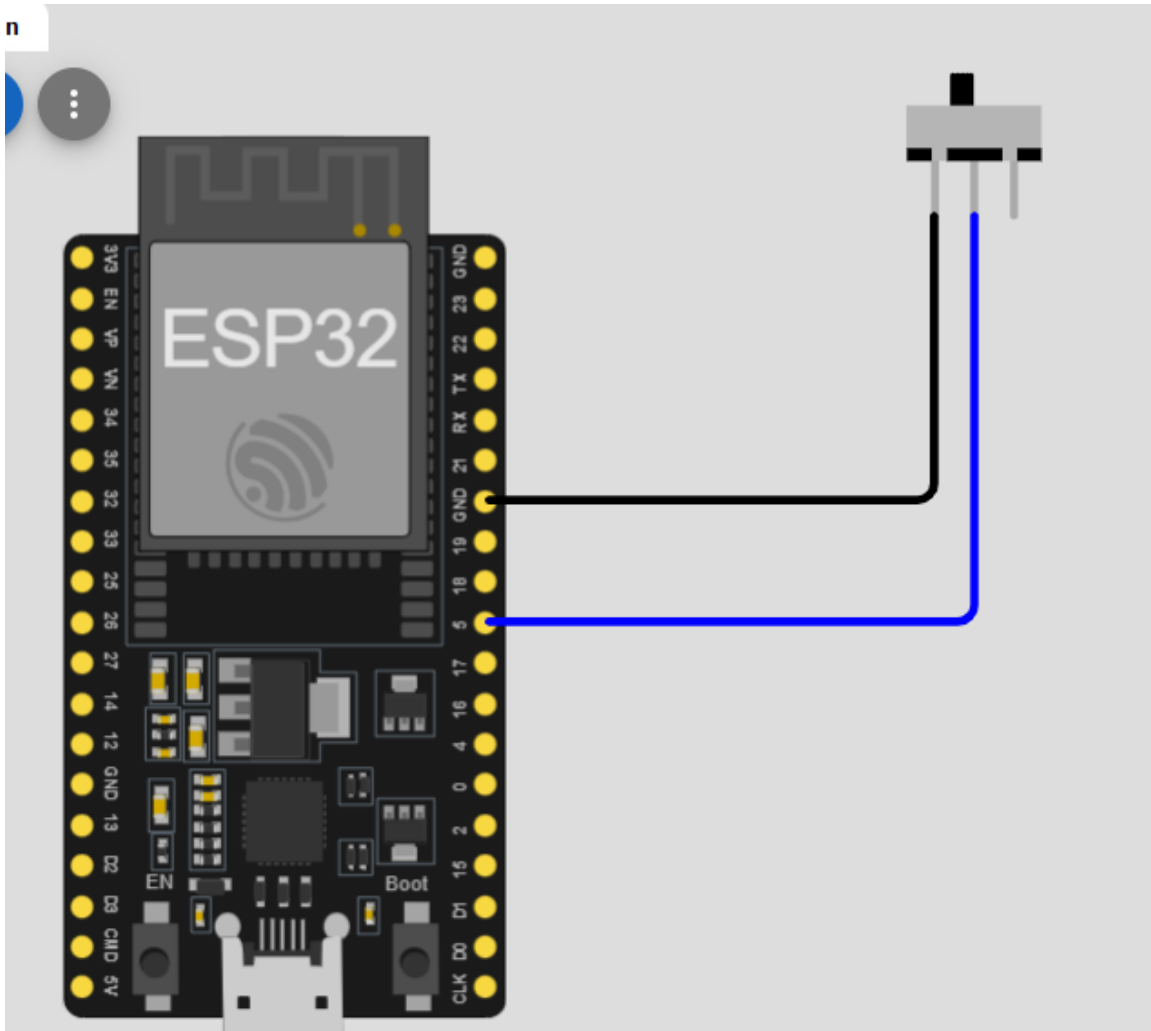
  // Check if the switch has been flipped to a new position
  if (currentSwitchState != lastSwitchState) {

    // If the pin is LOW, the switch closed the circuit to GND (ON)
    if (currentSwitchState == LOW) {
      Serial.println("Switch: ON");
    }
    // If the pin is HIGH, the switch opened the circuit (OFF)
    else {
      Serial.println("Switch: OFF");
    }

    // Save the current state for the next loop iteration
    lastSwitchState = currentSwitchState;

    // Small delay to prevent reading contact bounce while flipping
    delay(50);
  }
}
```

n

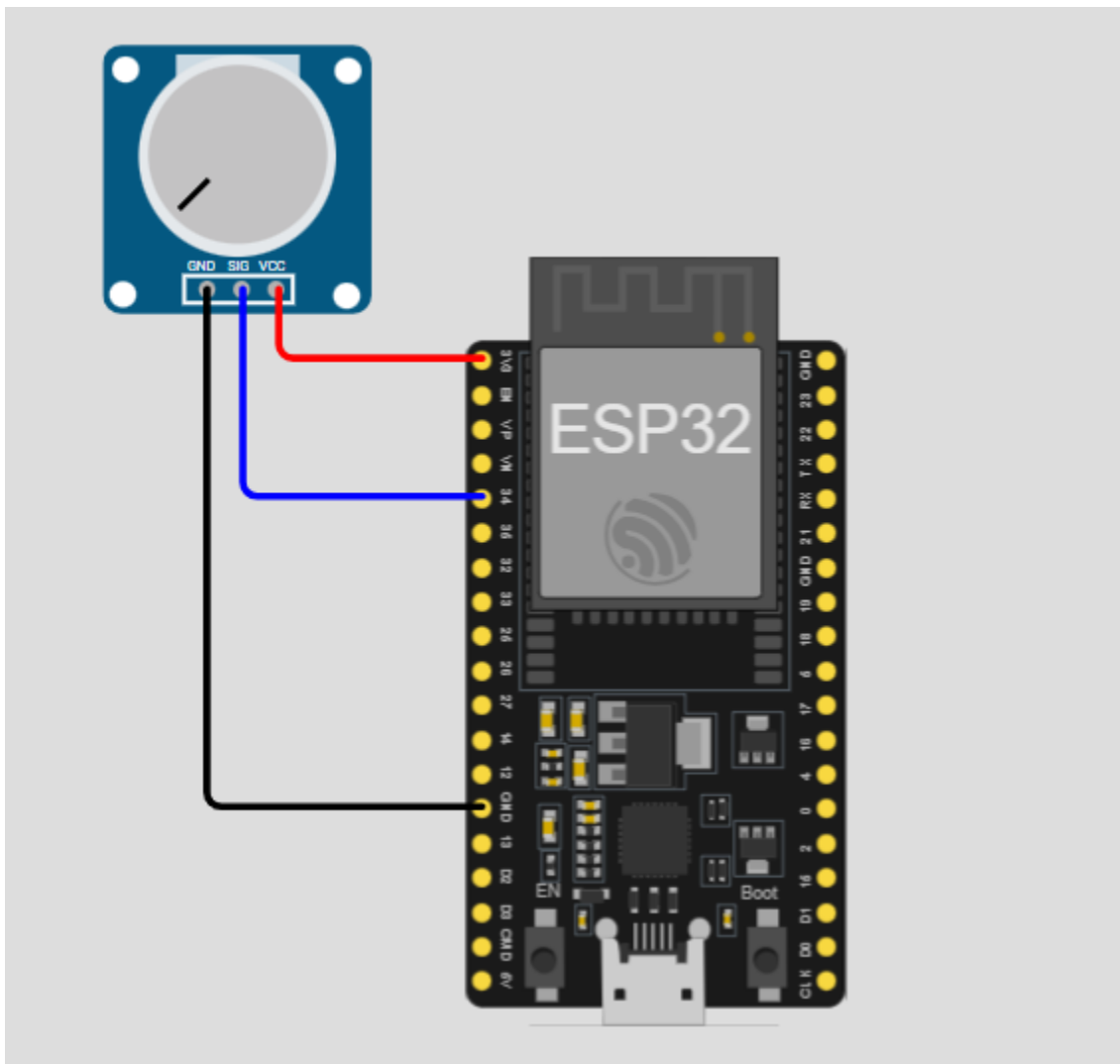


03.Potentiometer (Rotary)

- Write a program to:
 - Read the analog value from GPIO 34
 - Print values (0–4095) on Serial Monitor



Pins of device	Pins of ESP32
GND	GND
SIG	GPIO 34
VCC	3.3V



```
#include <Arduino.h>
```

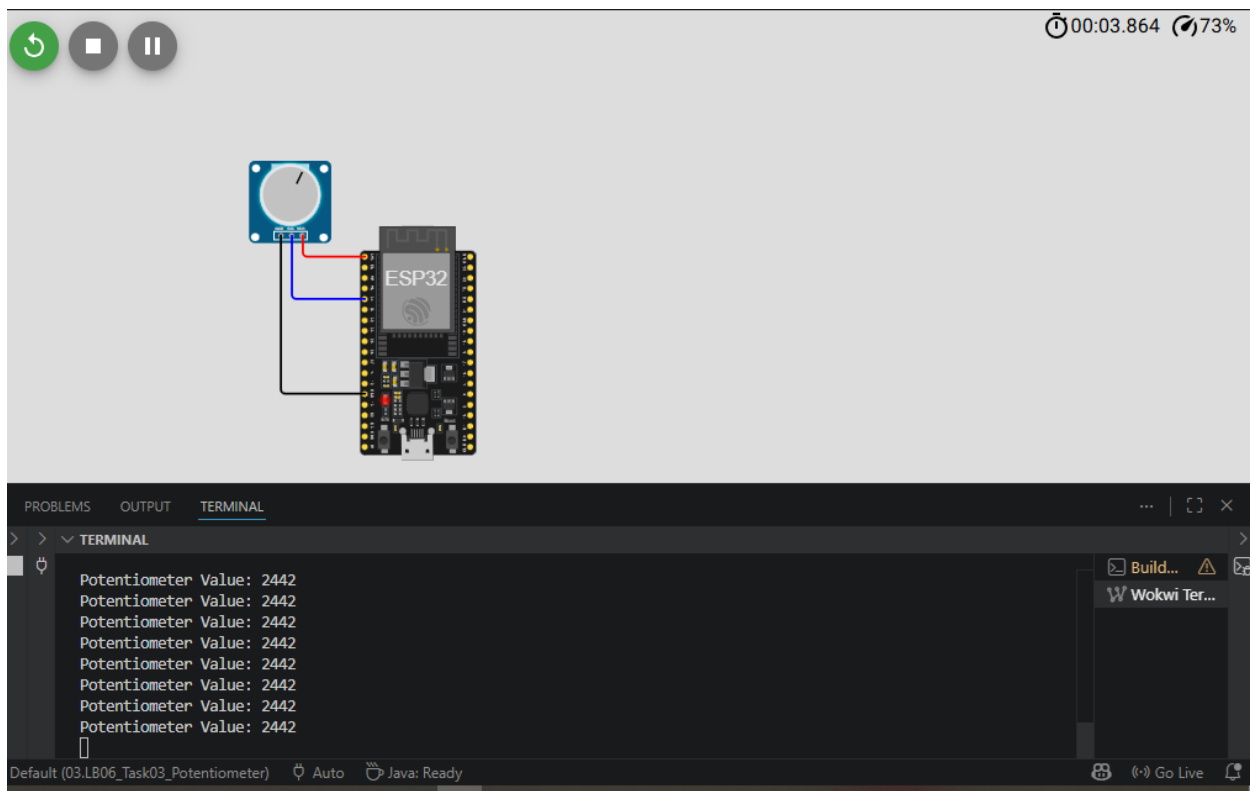
```
// Define the analog pin connected to the potentiometer middle pin
const int POT_PIN = 34; // GPIO 34 is an analog input pin (ADC1_CH6)

void setup() {
  // Initialize serial communication at 115200 baud
  Serial.begin(115200);
  Serial.println("Potentiometer Test");
  // Analog pins do not strictly require pinMode(),
  // but it's good practice to set it as INPUT.
  pinMode(POT_PIN, INPUT);
}

void loop() {
  // Read the analog value (0 to 4095)
  int potValue = analogRead(POT_PIN);

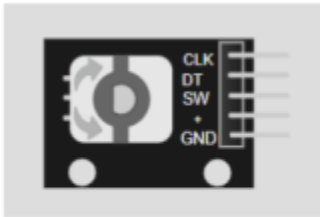
  // Print the value to the Serial Monitor
  Serial.print("Potentiometer Value: ");
  Serial.println(potValue);

  // Wait 100 milliseconds so the monitor doesn't scroll too fast
  delay(100);
}
```

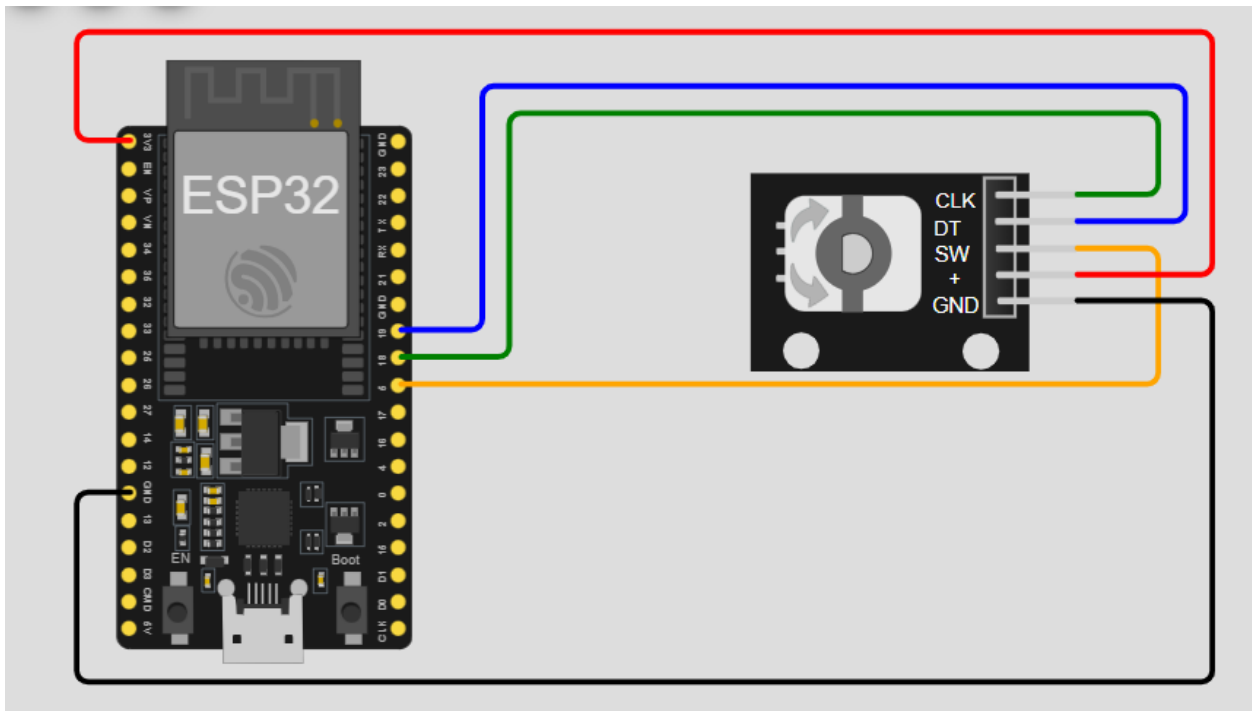


04.Rotary Encoder (KY-040)

- Write a program to:
 - Detect clockwise and anticlockwise rotation
 - Print direction and step count



Pins of device	Pins of ESP32
CLK	GPIO 18
DT	GPIO 19
SW	GPIO 5
VCC	3.3V
GND	GND



```
#include <Arduino.h>
// Define the encoder pins
const int CLK_PIN = 18; // Clock pin
const int DT_PIN = 19; // Data pin

int counter = 0;          // Stores the step count
int lastClkState;         // Stores the previous state of the CLK pin

void setup() {
```



```

// Initialize serial communication at 115200 baud
Serial.begin(115200);
Serial.println("Rotary Encoder (KY-040) Test");
// Set encoder pins as inputs
pinMode(CLK_PIN, INPUT);
pinMode(DT_PIN, INPUT);

// Read the initial state of the CLK pin
lastClkState = digitalRead(CLK_PIN);
}

void loop() {
  // Read the current state of the CLK pin
  int currentClkState = digitalRead(CLK_PIN);

  // Check if the knob has been turned (CLK state changed)
  if (currentClkState != lastClkState) {

    // If the CLK state changed, we check the DT state to find the direction
    // If CLK state is different from DT state, it's rotating Clockwise
    if (digitalRead(DT_PIN) != currentClkState) {
      counter++;
      Serial.print("Direction: Clockwise | ");
    }
    // Otherwise, it's rotating Counter-Clockwise
    else {
      counter--;
      Serial.print("Direction: Counter-Clockwise | ");
    }

    // Print the current step count
    Serial.print("Position: ");
    Serial.println(counter);
  }

  // Save the current CLK state for the next loop
  lastClkState = currentClkState;
}

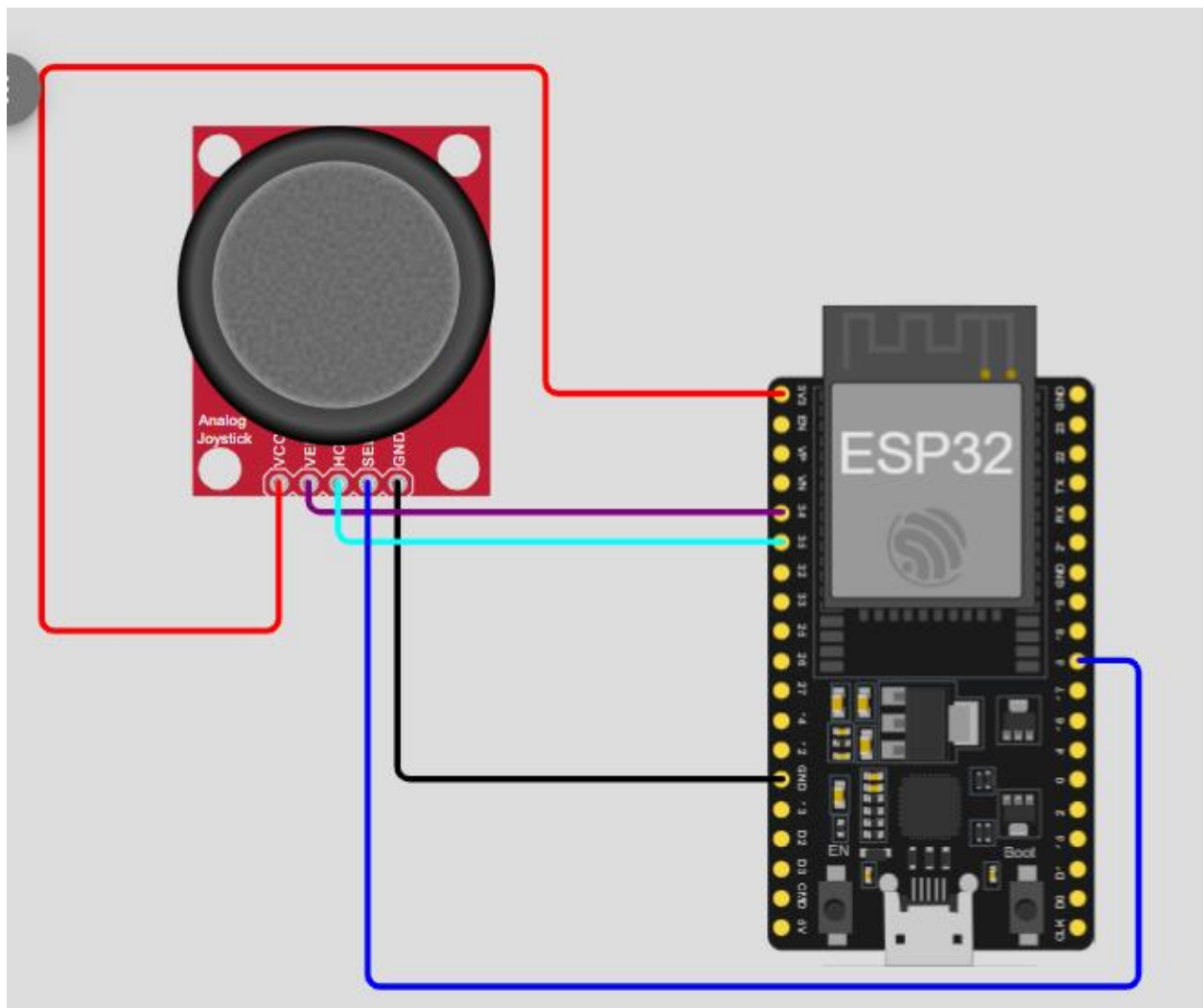
```

05. Analog Joystick

- Write a program to:
 - Read X-axis, Y-axis, and button press
 - Print values in Serial Monitor



Pins of device	Pins of ESP32
VCC	3.3V
VERT	GPIO 35
HORZ	GPIO 34
SEL	GPIO 5
GND	GND



```
#include <Arduino.h>
```

```

// Define the joystick pins
const int X_PIN = 34; // Analog pin for X-axis
const int Y_PIN = 35; // Analog pin for Y-axis
const int SW_PIN = 5; // Digital pin for the push-button

void setup() {
    // Initialize serial communication at 115200 baud
    Serial.begin(115200);
    Serial.println("Analog Joystick Test");
    // Configure axis pins as inputs
    pinMode(X_PIN, INPUT);
    pinMode(Y_PIN, INPUT);

    // Configure the switch pin with internal pull-up resistor
    pinMode(SW_PIN, INPUT_PULLUP);
}

void loop() {
    // Read the analog values (0 to 4095)
    int xValue = analogRead(X_PIN);
    int yValue = analogRead(Y_PIN);

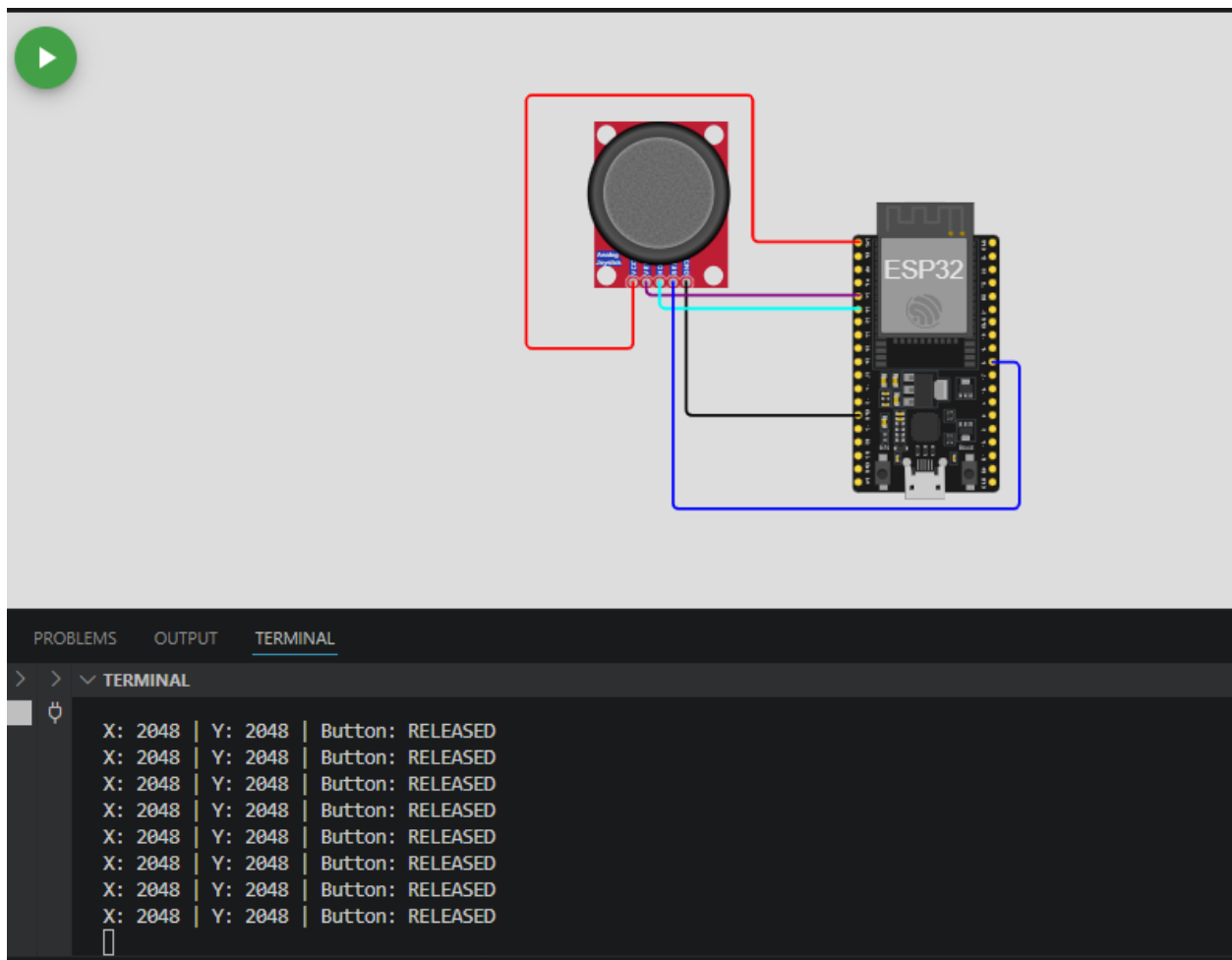
    // Read the digital button state (LOW = Pressed, HIGH = Released)
    int buttonState = digitalRead(SW_PIN);

    // Print the data to the Serial Monitor
    Serial.print("X: ");
    Serial.print(xValue);
    Serial.print(" | Y: ");
    Serial.print(yValue);
    Serial.print(" | Button: ");

    if (buttonState == LOW) {
        Serial.println("PRESSED");
    } else {
        Serial.println("RELEASED");
    }

    // Wait 150 milliseconds so the text is readable
    delay(150);
}

```

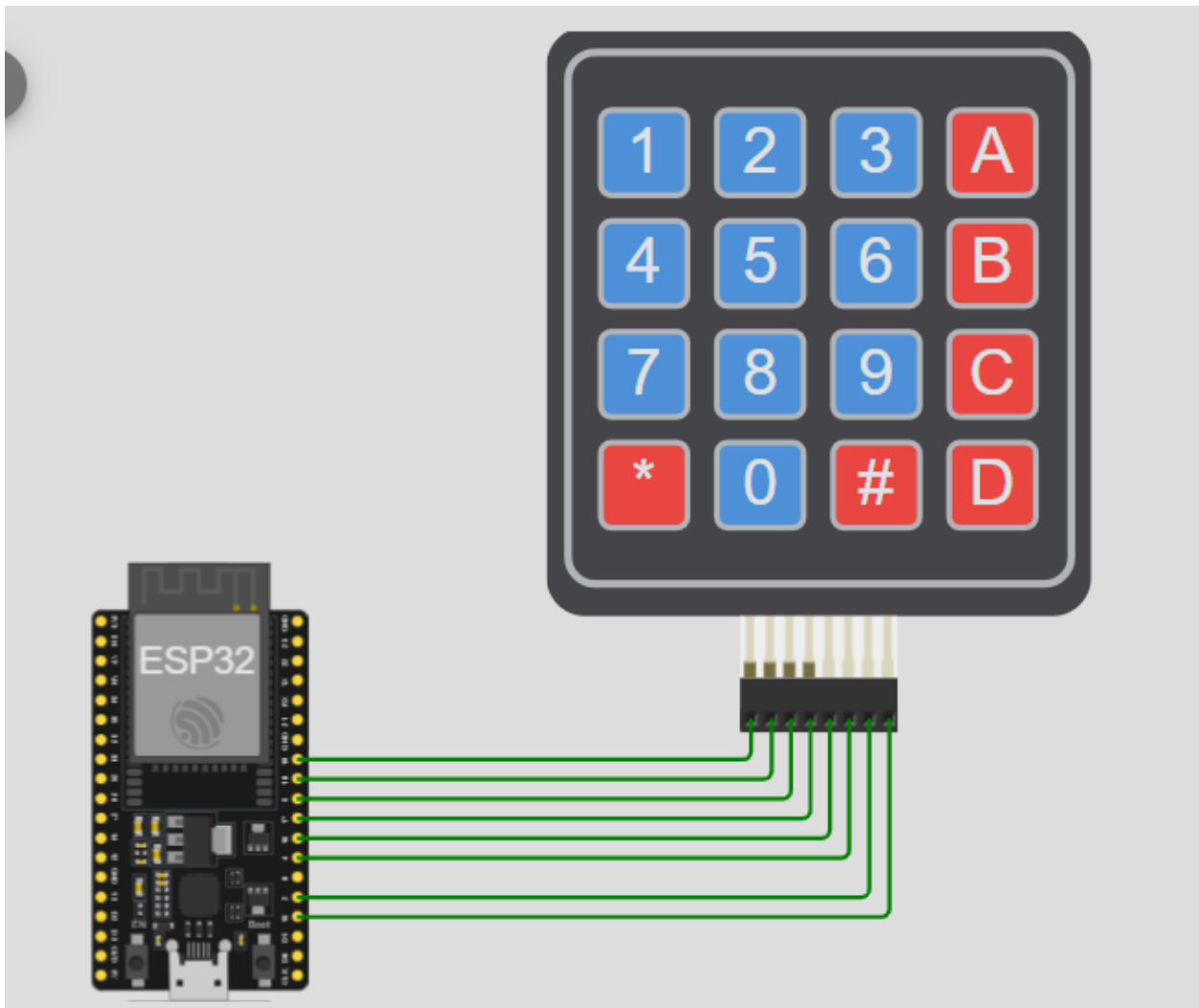


06.Keypad

- Write a program to:
 - Detect key presses
 - Display pressed key on Serial Monitor



Pins of device	Pins of ESP32
R1	GPIO 19
R2	GPIO 18
R3	GPIO 5
R4	GPIO 17
C1	GPIO 16
C2	GPIO 4
C3	GPIO 2
C4	GPIO 15



```
#include <Arduino.h>
#include <Keypad.h>

// Define the number of rows and columns on your keypad
const byte ROWS = 4;
const byte COLS = 4;

// Define the symbols on the buttons of the keypads
char hexaKeys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

// Connect the ESP32 GPIO pins to the row pins of the keypad
byte rowPins[ROWS] = {19, 18, 5, 17};
```

```

// Connect the ESP32 GPIO pins to the column pins of the keypad
byte colPins[COLS] = {16, 4, 2, 15};

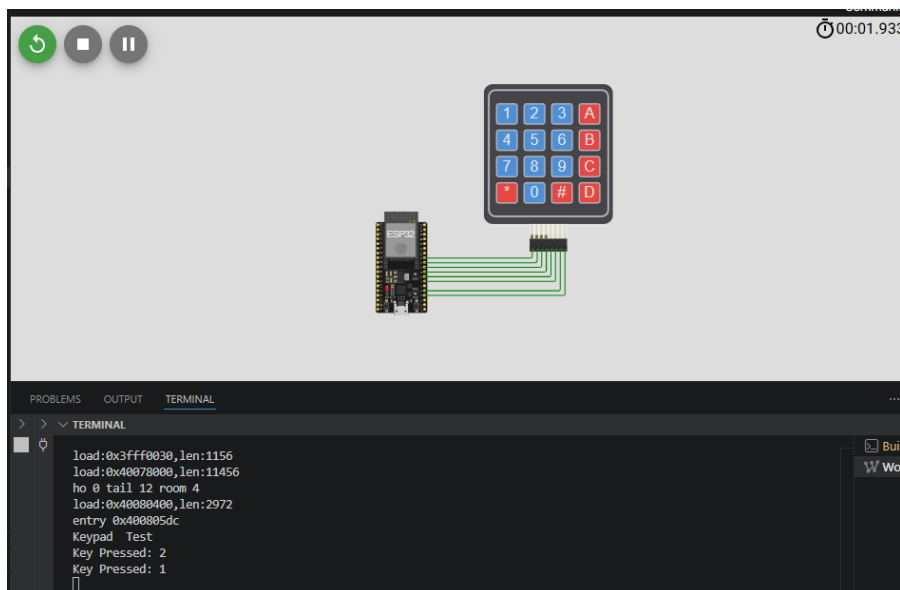
// Initialize an instance of class NewKeypad
Keypad customKeypad = Keypad(makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS);

void setup() {
    // Initialize serial communication at 115200 baud
    Serial.begin(115200);
    Serial.println("Keypad Test");
}

void loop() {
    // Check if a key is pressed
    char customKey = customKeypad.getKey();

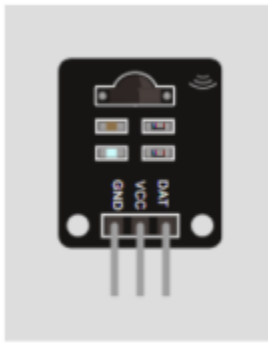
    // If a key was pressed, print it to the Serial Monitor
    if (customKey) {
        Serial.print("Key Pressed: ");
        Serial.println(customKey);
    }
}

```

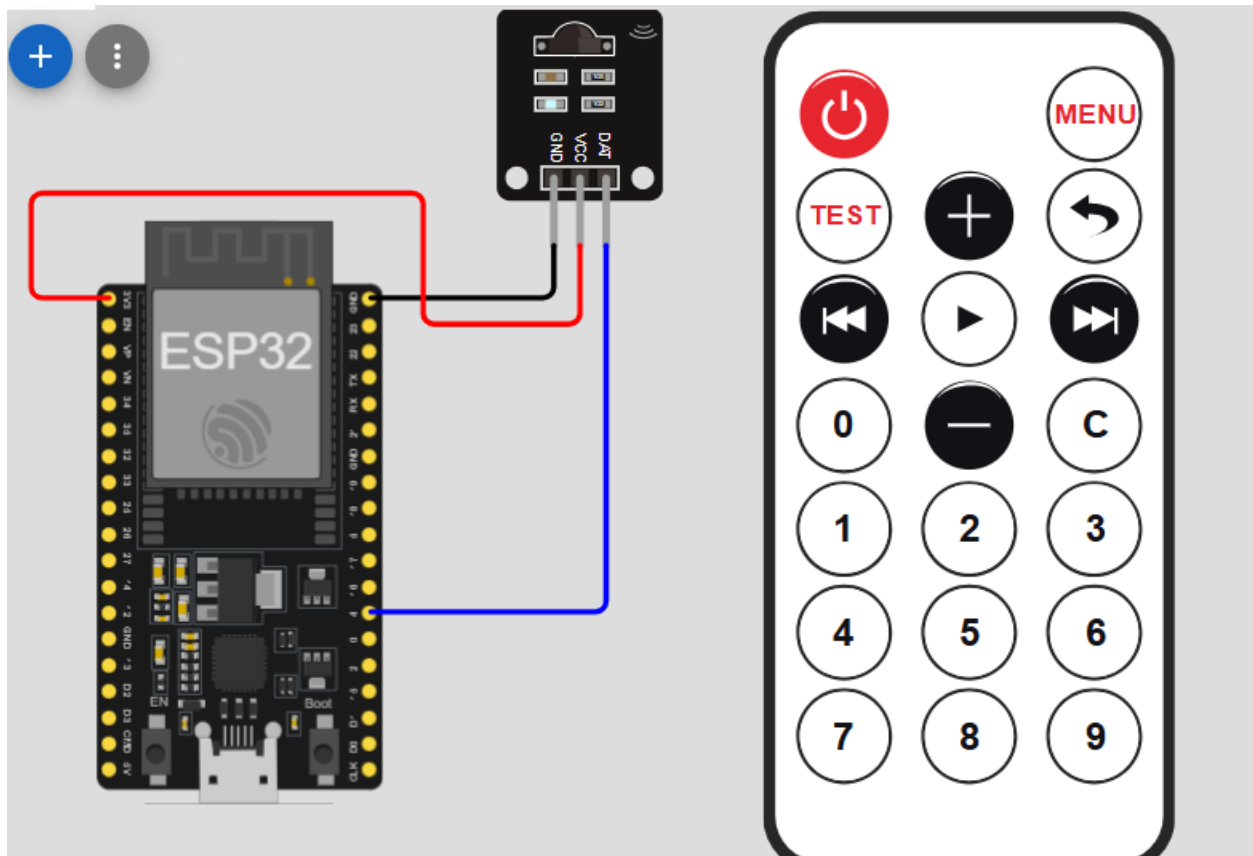


07.IR Receiver

- Write a program to:
 - Receive IR remote signals
 - Print received HEX code



Pins of device	Pins of ESP32
GND	GND
VCC	3.3V
DAT	GPIO 4



```
#include <Arduino.h>
#define IR_RECEIVE_PIN 4 // Define the pin connected to the IR receiver output

// The IRremote Library requires this macro definition BEFORE including the
// header
#define DECODE_NEC          // Optional: Saves memory by only enabling specific
// protocols
#include <IRremote.hpp>

void setup() {
  // Initialize serial communication at 115200 baud
```

```

Serial.begin(115200);
Serial.println("IR Receiver Test");
// Start the receiver and enable visual feedback (blinks onboard LED if
applicable)
IrReceiver.begin(IR_RECEIVE_PIN, ENABLE_LED_FEEDBACK);
Serial.println("IR Receiver is ready. Point a remote and press a button!");
}

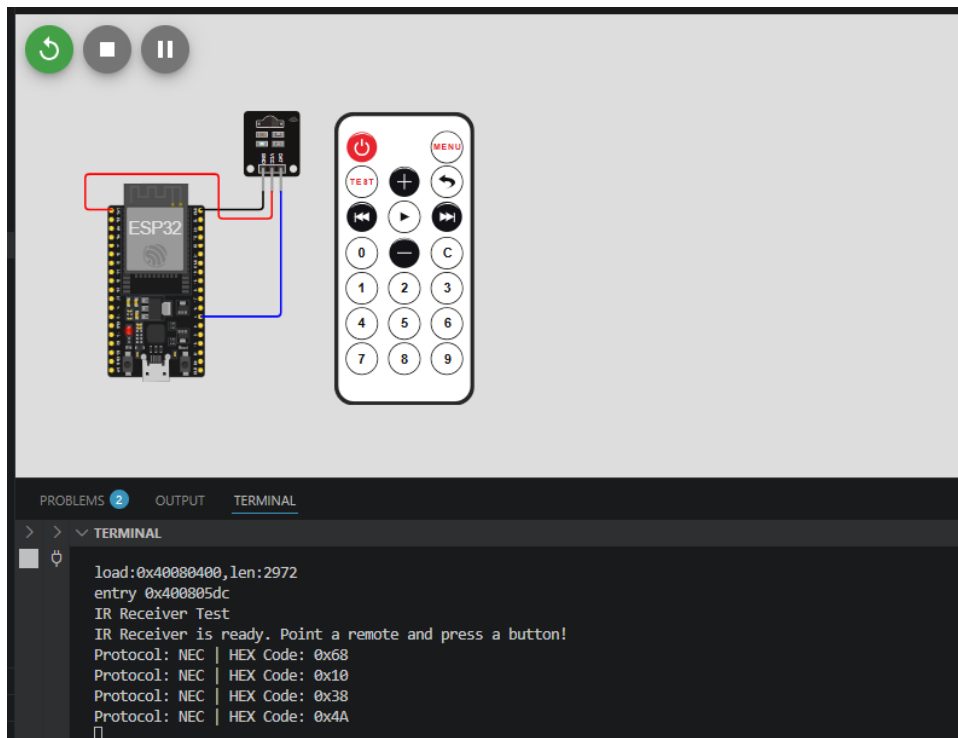
void loop() {
  // Check if a new IR signal has been received and decoded
  if (IrReceiver.decode()) {

    // Print the command in Hexadecimal format
    Serial.print("Protocol: ");
    Serial.print(IrReceiver.getProtocolString()); // Prints the protocol name
    (e.g., NEC, SONY)

    Serial.print(" | HEX Code: 0x");
    Serial.println(IrReceiver.decodedIRData.command, HEX); // Prints the unique
    button HEX value

    // Resume receiving to prepare for the next button press
    IrReceiver.resume();
  }
}

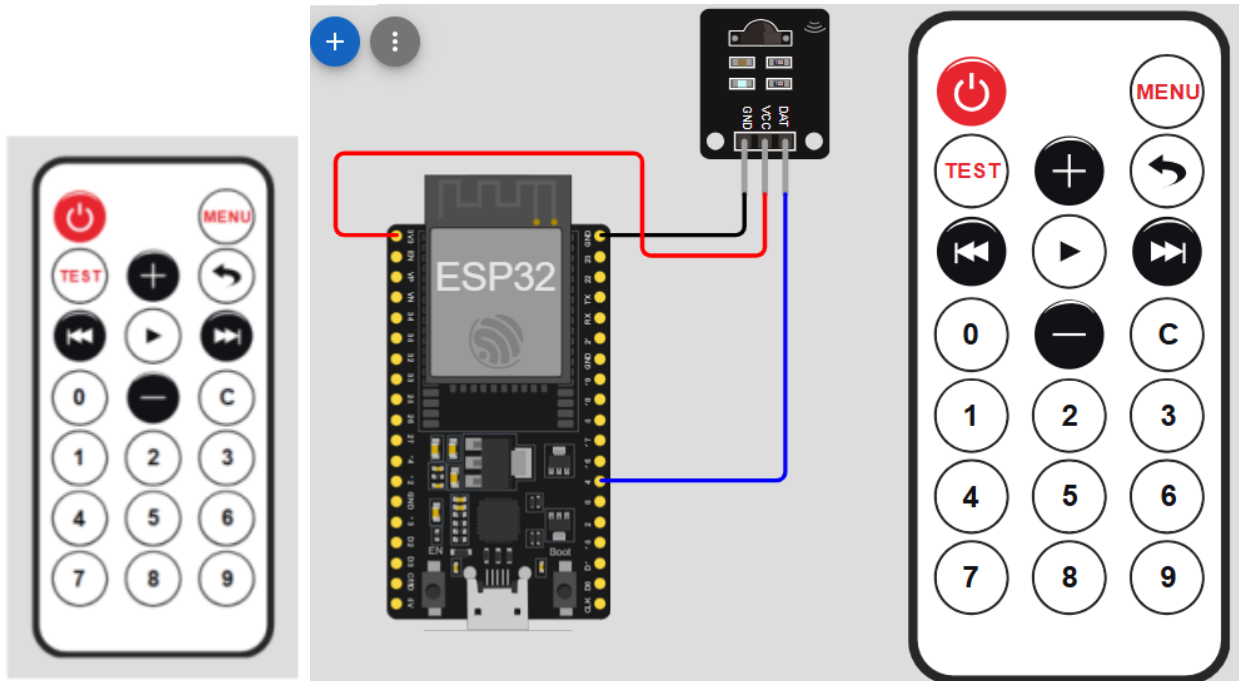
```



Output Devices

01.IR Remote

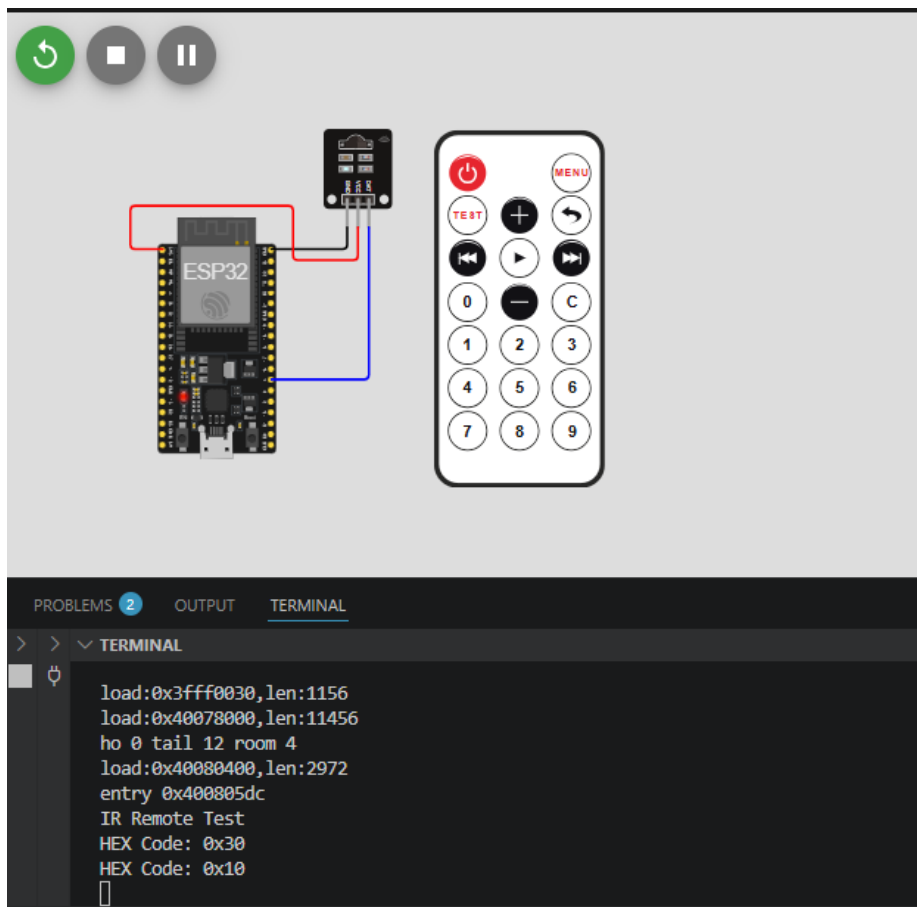
- Observe how:
 - Button press sends IR signal
 - Signal is decoded by ESP32



```
#include <Arduino.h>
#define IR_RECEIVE_PIN 4
#include <IRremote.hpp>

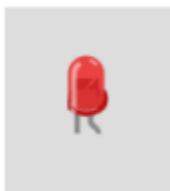
void setup() {
  Serial.begin(115200);
  Serial.println("IR Remote Test");
  IrReceiver.begin(IR_RECEIVE_PIN, ENABLE_LED_FEEDBACK);
}

void loop() {
  if (IrReceiver.decode()) {
    Serial.print("HEX Code: 0x");
    Serial.println(IrReceiver.decodedIRData.command, HEX);
    IrReceiver.resume();
  }
}
```

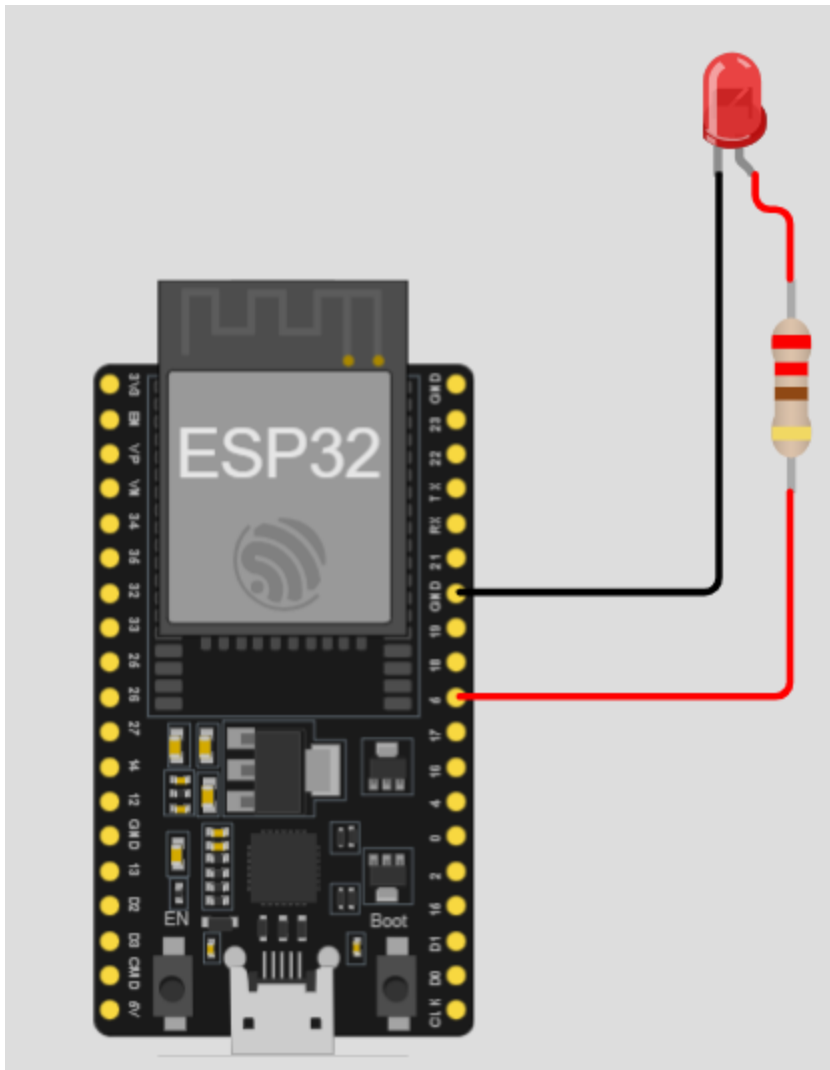


02.LED

- Write a program to:
 - Blink LED with 1-second delay



Pins of device	Pins of ESP32
C	GND
A	GPIO 5 (via 220Ω)

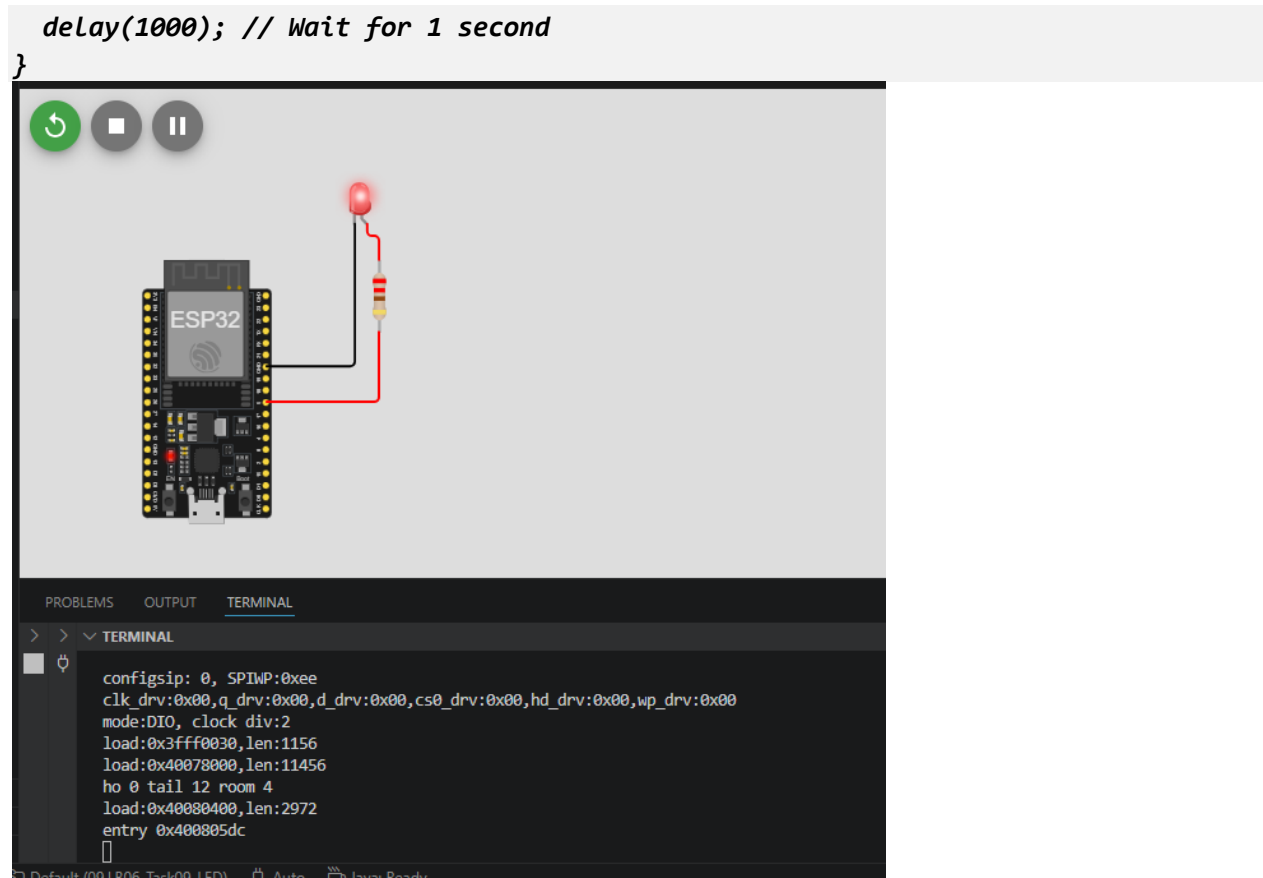


```
#include <Arduino.h>
const int LED_PIN = 2; // GPIO 5 is also the built-in blue LED on most ESP32 boards

void setup() {
  // Configure the LED pin as an OUTPUT
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  // Turn the LED ON (HIGH voltage level)
  digitalWrite(LED_PIN, HIGH);
  delay(1000); // Wait for 1 second (1000 milliseconds)

  // Turn the LED OFF (LOW voltage level)
  digitalWrite(LED_PIN, LOW);
}
```

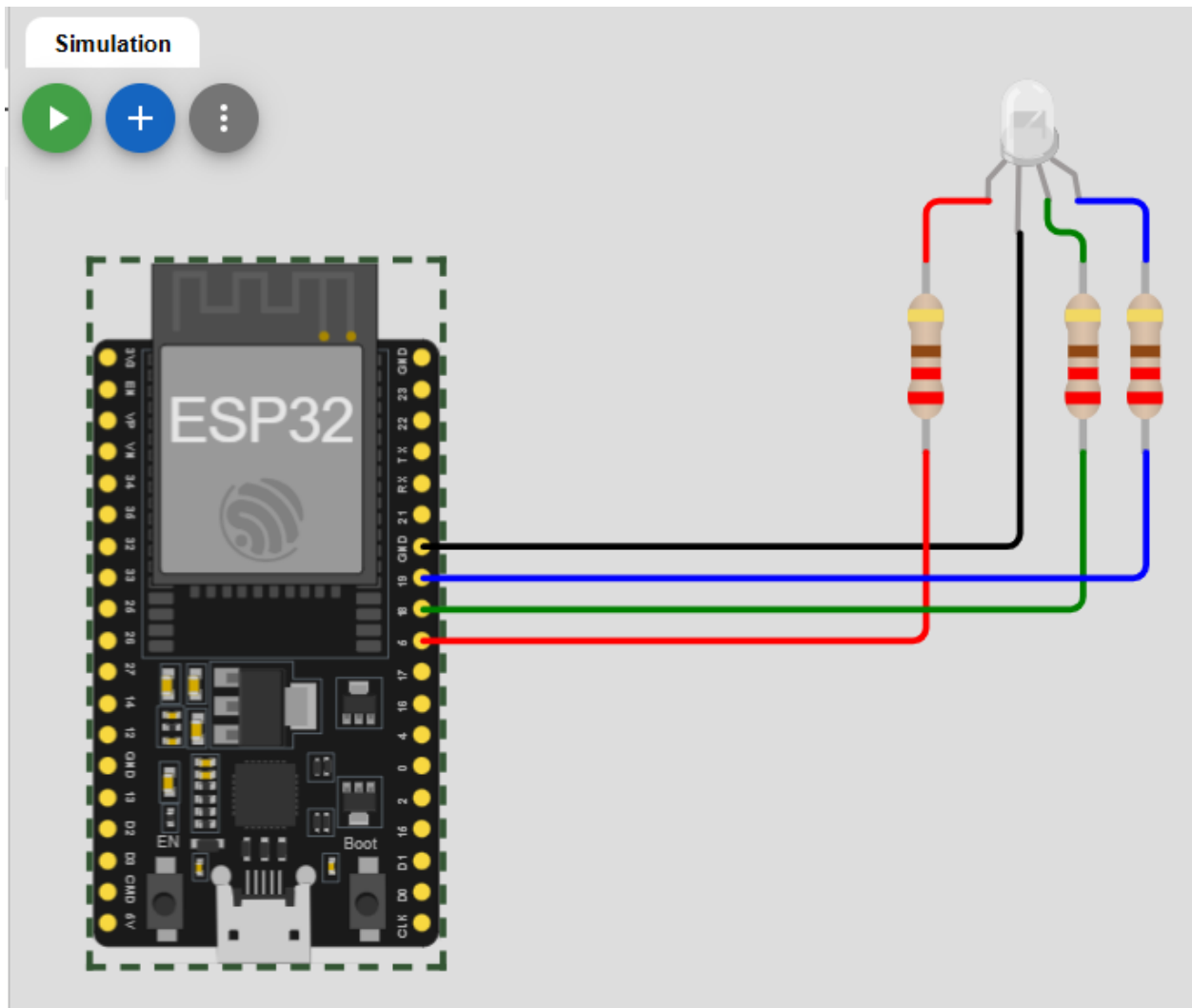


03.RGB LED

- Write a program to:
 - Display Red, Green, Blue, and White colors



Pins of device	Pins of ESP32
R	GPIO 5 (via 220Ω)
COM	GND
G	GPIO 18 (via 220Ω)
B	GPIO 19 (via 220Ω)



```
#include <Arduino.h>

// Define the GPIO pins for each color channel
const int RED_PIN = 5;
const int GREEN_PIN = 18;
const int BLUE_PIN = 19;

void setColor(int redState, int greenState, int blueState);

void setup() {
  // Set all color pins as OUTPUT
  Serial.begin(115200);
  Serial.println("RGB LED Test");
  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  pinMode(BLUE_PIN, OUTPUT);
}
```

```

void loop() {
  // 1. Display RED
  setColor(HIGH, LOW, LOW);
  delay(1000);

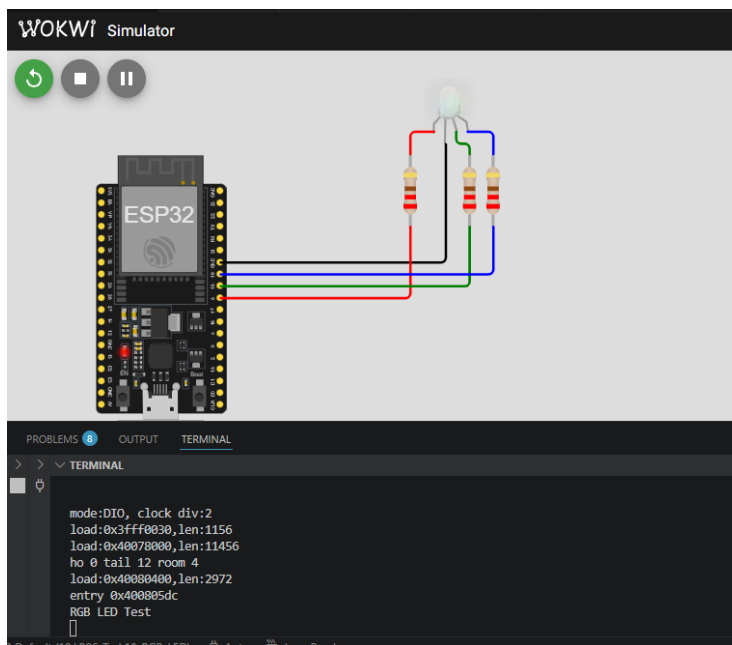
  // 2. Display GREEN
  setColor(LOW, HIGH, LOW);
  delay(1000);

  // 3. Display BLUE
  setColor(LOW, LOW, HIGH);
  delay(1000);

  // 4. Display WHITE (Red + Green + Blue mixed together)
  setColor(HIGH, HIGH, HIGH);
  delay(1000);
}

// Helper function to easily set the pin states
void setColor(int redState, int greenState, int blueState) {
  digitalWrite(RED_PIN, redState);
  digitalWrite(GREEN_PIN, greenState);
  digitalWrite(BLUE_PIN, blueState);
}

```



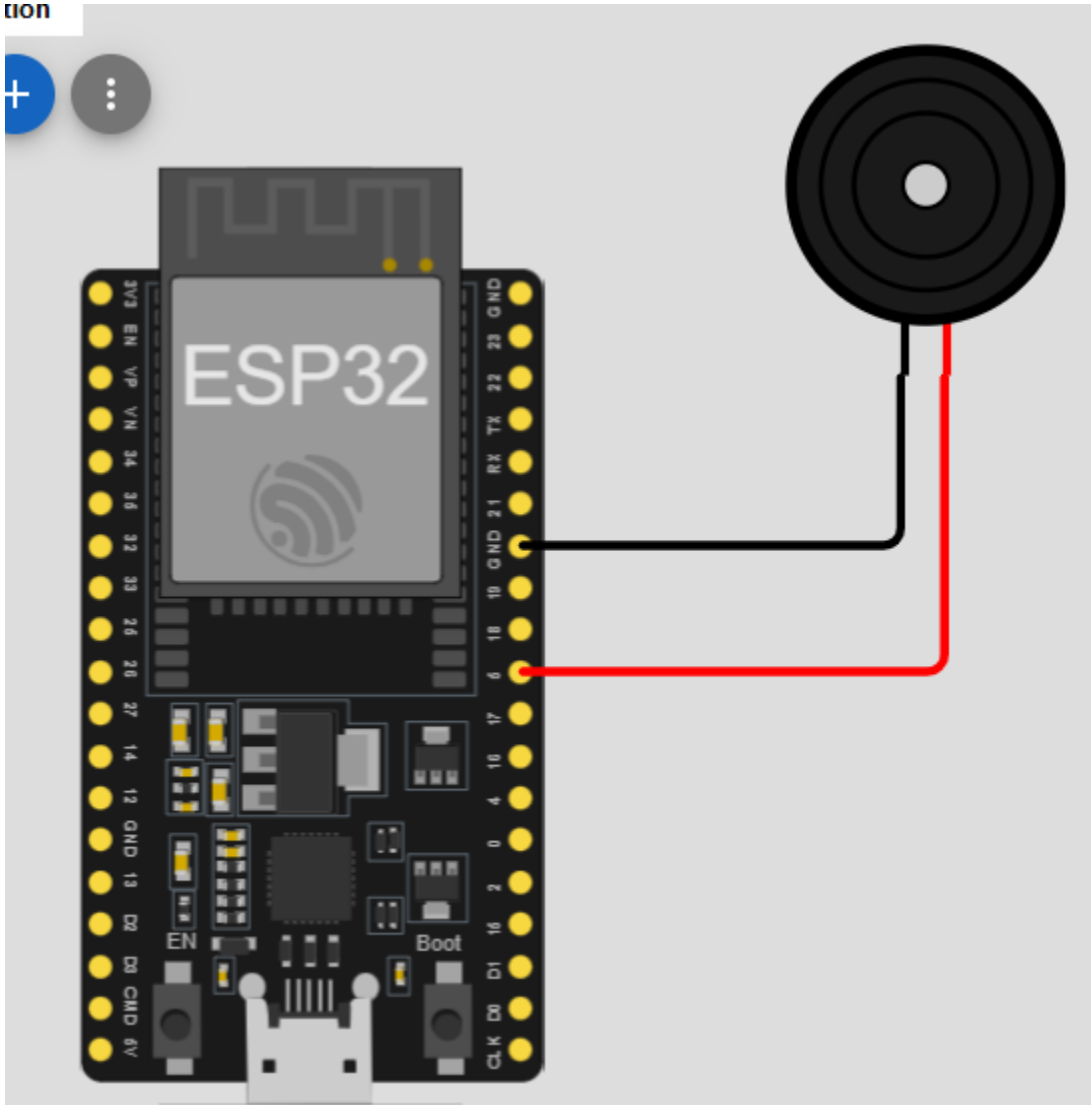
04.Buzzer

- Write a program to:
 - Generate beep sound using GPIO



Pins of device	Pins of ESP32
1	GND
2	GPIO 5

tion



```
#include <Arduino.h>
// Define the GPIO pin connected to the buzzer
const int BUZZER_PIN = 5; // You can use GPIO 12, 13, 14, etc.
```

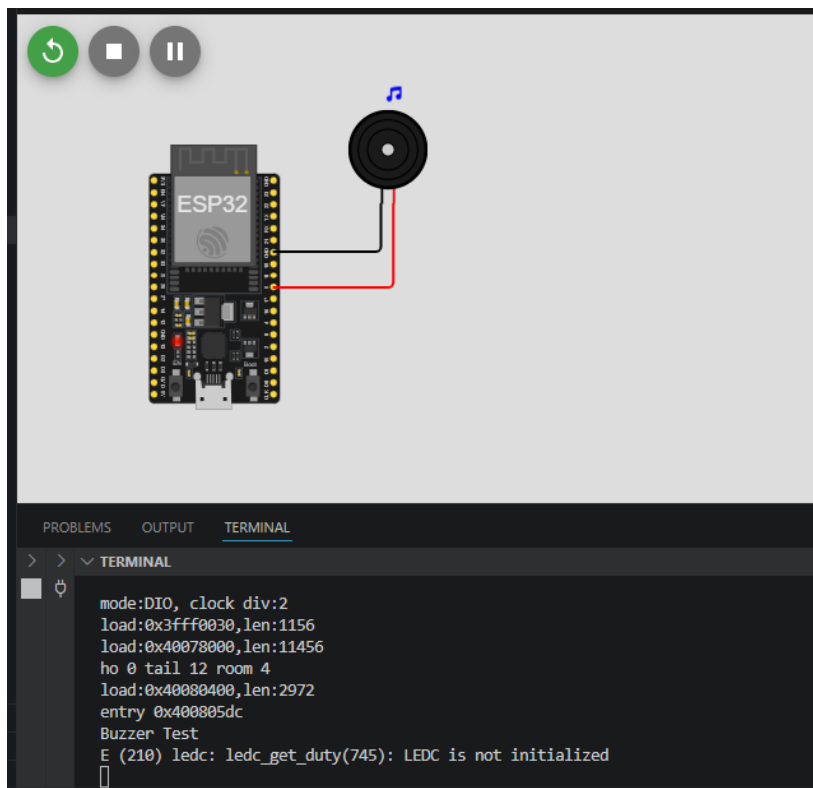
```

void setup() {
  // Configure the buzzer pin as an OUTPUT
  Serial.begin(115200);
  Serial.println("Buzzer Test");
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  // --- Generate the Beep Sound ---
  // tone(pin, frequency, duration)
  // 2000Hz frequency creates a standard crisp beep
  tone(BUZZER_PIN, 2000, 200);

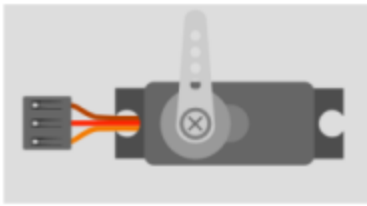
  // Wait for 1 second before the next beep
  // Note: tone() doesn't pause the code, so we use delay()
  delay(1000);
}

```

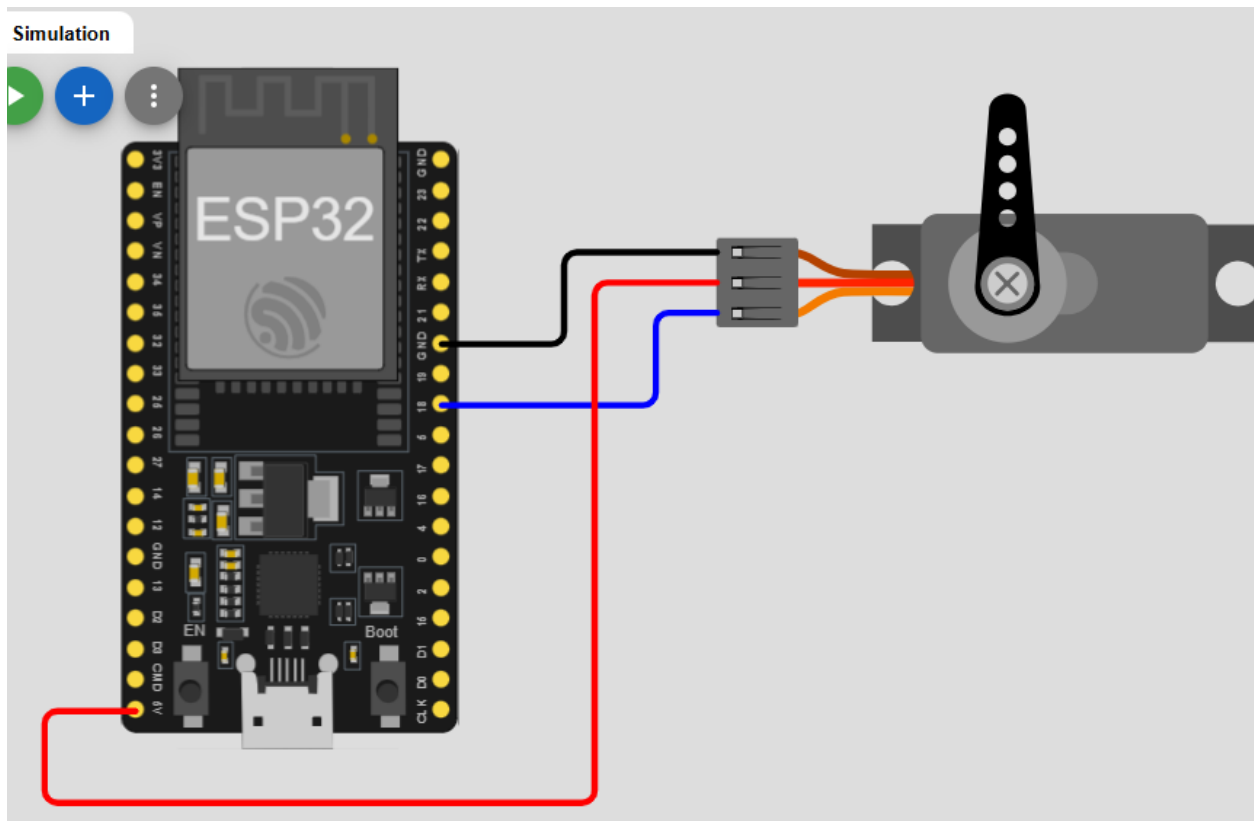


05.Servo Motor

- Write a program to:
 - Rotate servo from $0^{\circ} \rightarrow 90^{\circ} \rightarrow 180^{\circ}$



Pins of device	Pins of ESP32
GND	GND
V+	5V
PWM	GPIO 18



```
#include <Arduino.h>
#include <ESP32Servo.h>

// Create a servo object
Servo myServo;

// Define the GPIO pin connected to the servo signal wire
const int SERVO_PIN = 18;

void setup() {
  // Allow allocation of all timers for servo PWM control
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  ESP32PWM::allocateTimer(2);
}
```

```

ESP32PWM::allocateTimer(3);
Serial.begin(115200);
Serial.println("Servo Motor");

// Set the PWM frequency for a standard servo (50Hz)
myServo.setPeriodHertz(50);

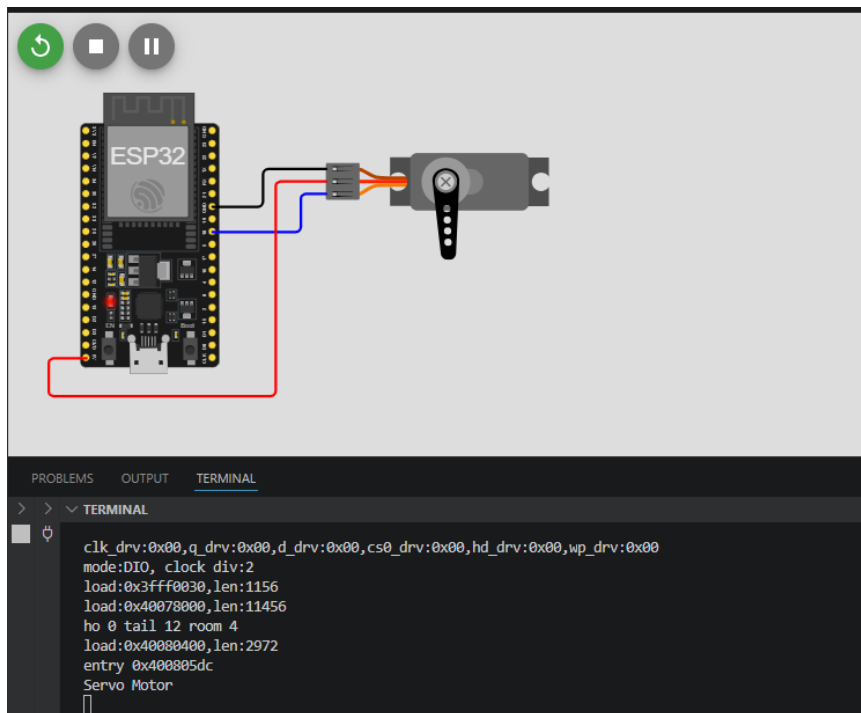
// Attach the servo object to the pin
myServo.attach(SERVO_PIN, 500, 2400); // Pulse width range in microseconds
}

void loop() {
    // Move to 0 degrees
    myServo.write(0);
    delay(1000); // Wait 1 second for the motor to reach the position

    // Move to 90 degrees
    myServo.write(90);
    delay(1000);

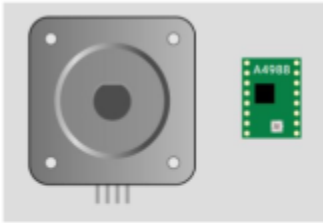
    // Move to 180 degrees
    myServo.write(180);
    delay(1000);
}

```

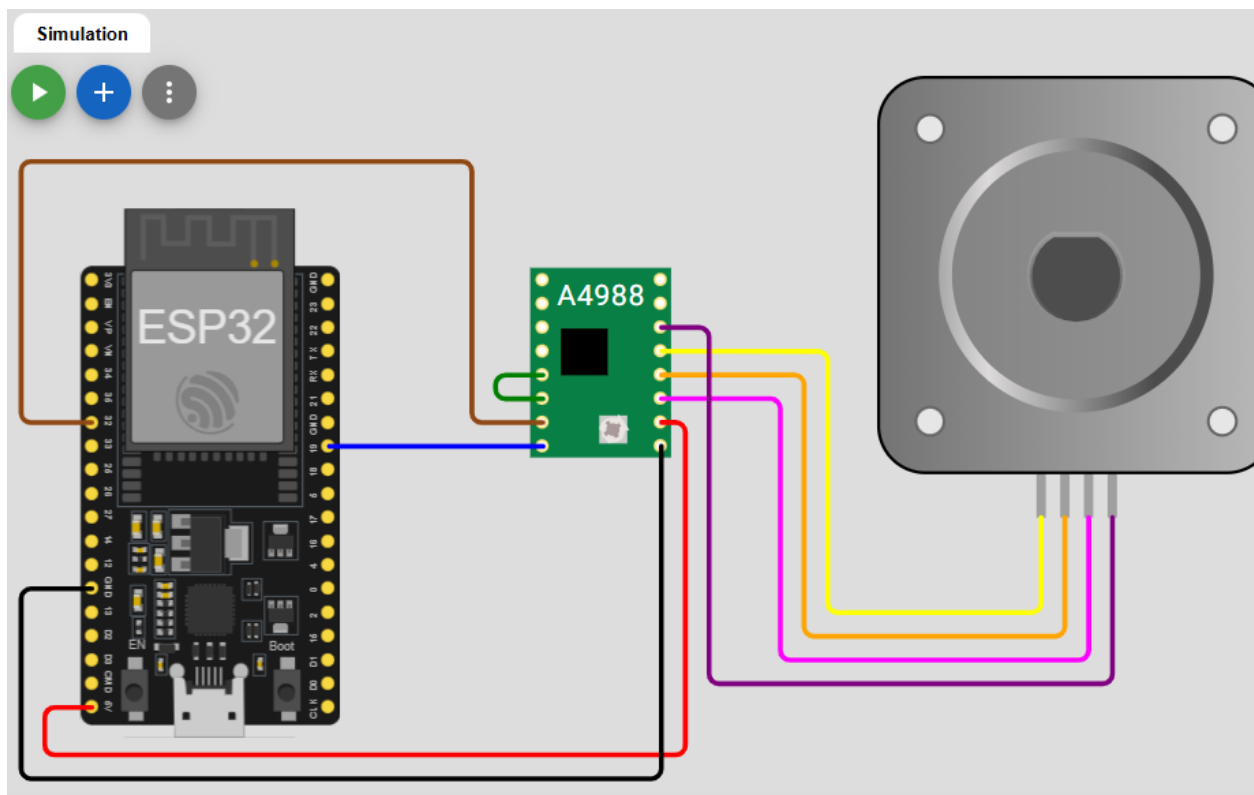


06.Bipolar Stepper Motor + A4988 Driver

- Write a program to:
 - Rotate stepper motor clockwise and anticlockwise



Pins of Stepper motor	Pins of A4988 driver	Pins of ESP32
A-	2A	
A+	1A	
B+	1B	
B-	2B	
	RESET	Connect Together
	SLEEP	
	STEP	GPIO 32
	DIR	GPIO 19
	VDD	5V
	GND	GND



```
#include <Arduino.h>
const int STEP_PIN = 32; // GPIO 32 sends the step pulses
const int DIR_PIN = 19; // GPIO 19 sets the rotation direction
```

```

// Define the steps per revolution for your motor
// (Standard NEMA 17 motors usually have 200 steps per full revolution)
const int STEPS_PER_REV = 200;

void setup() {
  // Configure control pins as outputs
  pinMode(STEP_PIN, OUTPUT);
  pinMode(DIR_PIN, OUTPUT);
  Serial.begin(115200);
  Serial.println("Bipolar Stepper Motor + A4988 Driver  Test");
}

void loop() {
  // 1. Rotate Clockwise
  digitalWrite(DIR_PIN, HIGH); // Set direction to HIGH (Clockwise)

  // Spin one full revolution
  for (int i = 0; i < STEPS_PER_REV; i++) {
    digitalWrite(STEP_PIN, HIGH);
    delayMicroseconds(1000); // Speeds up or slows down the pulse rate
    digitalWrite(STEP_PIN, LOW);
    delayMicroseconds(1000);
  }

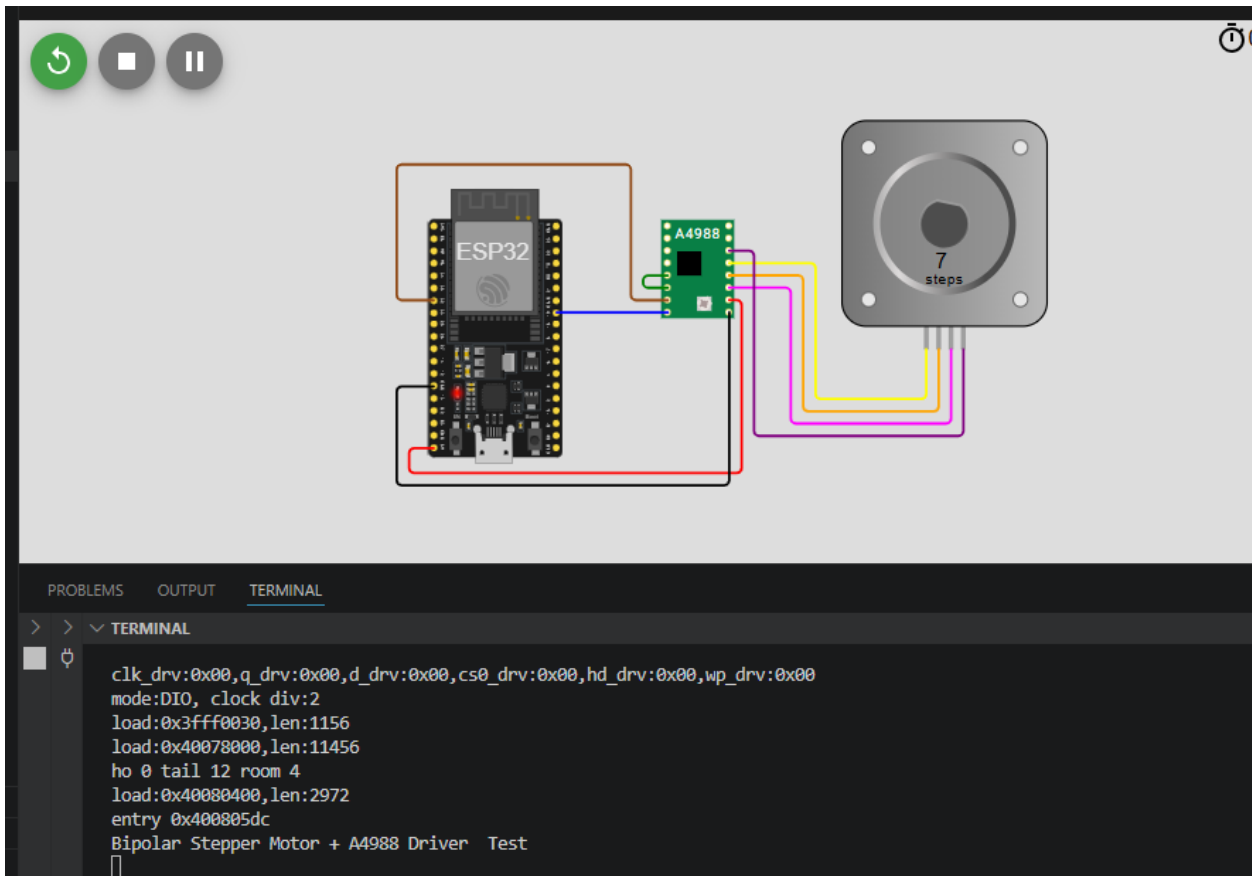
  delay(1000); // Pause for 1 second

  // 2. Rotate Counter-Clockwise
  digitalWrite(DIR_PIN, LOW); // Set direction to LOW (Counter-Clockwise)

  // Spin one full revolution back
  for (int i = 0; i < STEPS_PER_REV; i++) {
    digitalWrite(STEP_PIN, HIGH);
    delayMicroseconds(1000);
    digitalWrite(STEP_PIN, LOW);
    delayMicroseconds(1000);
  }

  delay(1000); // Pause for 1 second
}

```

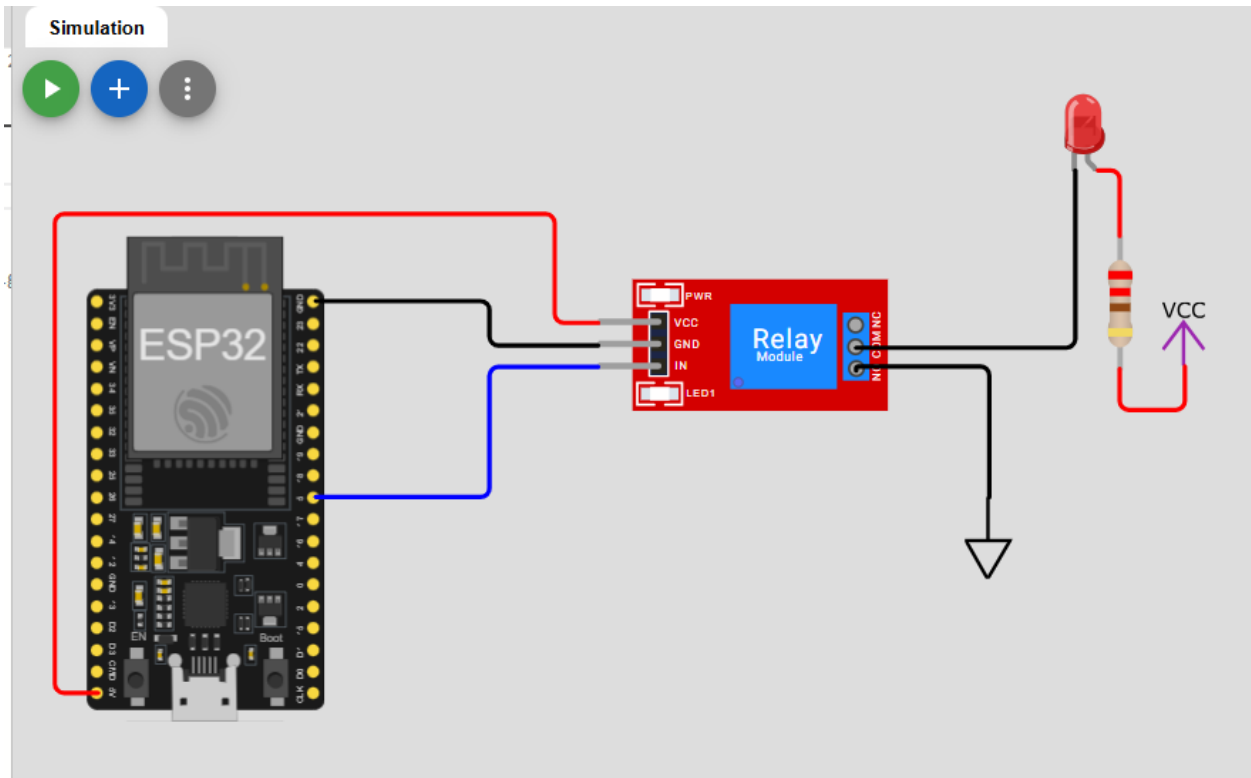


07.Relay Module

- Write a program to:
 - Turn relay ON and OFF using ESP32



	Pins of device	Pins of ESP32
Input	VCC	5V
	GND	GND
	IN	GPIO 5
Output	NC	-
	COM	GND PIN of Output device
	NO	GND

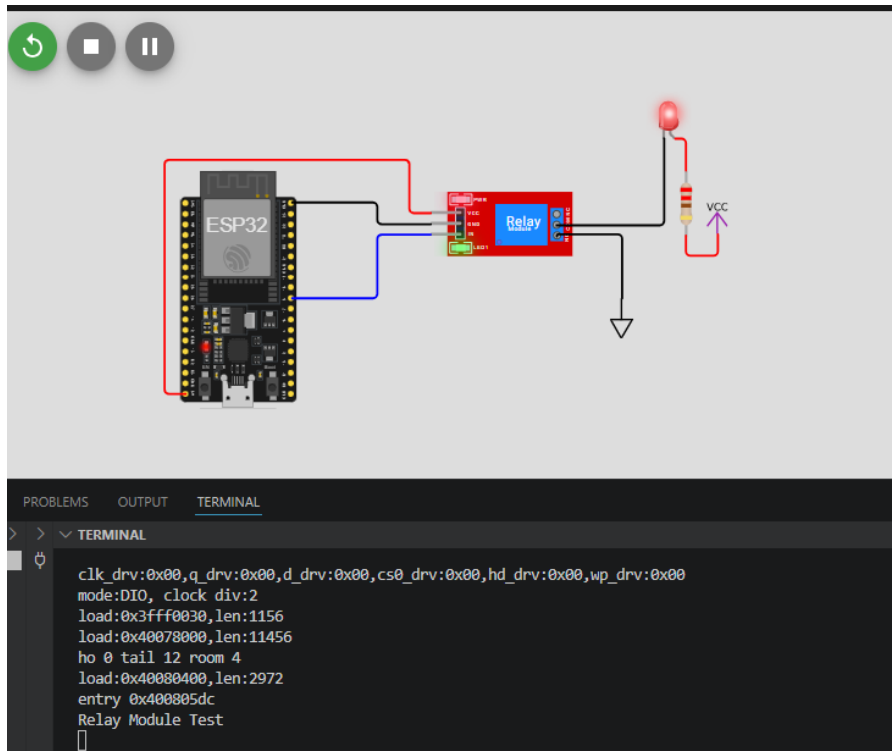


```
#include <Arduino.h>
const int RELAY_PIN = 5; // You can use GPIO 25, 26, 27, etc.

void setup() {
  // Configure the relay pin as an OUTPUT
  pinMode(RELAY_PIN, OUTPUT);
  Serial.begin(115200);
  Serial.println("Relay Module Test");
  // Optional: Set initial state to OFF.
  // For Active-Low relays, HIGH = OFF. For Active-High relays, LOW = OFF.
  digitalWrite(RELAY_PIN, LOW);
}

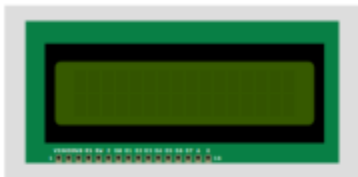
void loop() {
  // Turn the relay ON
  digitalWrite(RELAY_PIN, HIGH);
  delay(2000); // Wait for 2 seconds

  // Turn the relay OFF
  digitalWrite(RELAY_PIN, LOW);
  delay(2000); // Wait for 2 seconds
}
```

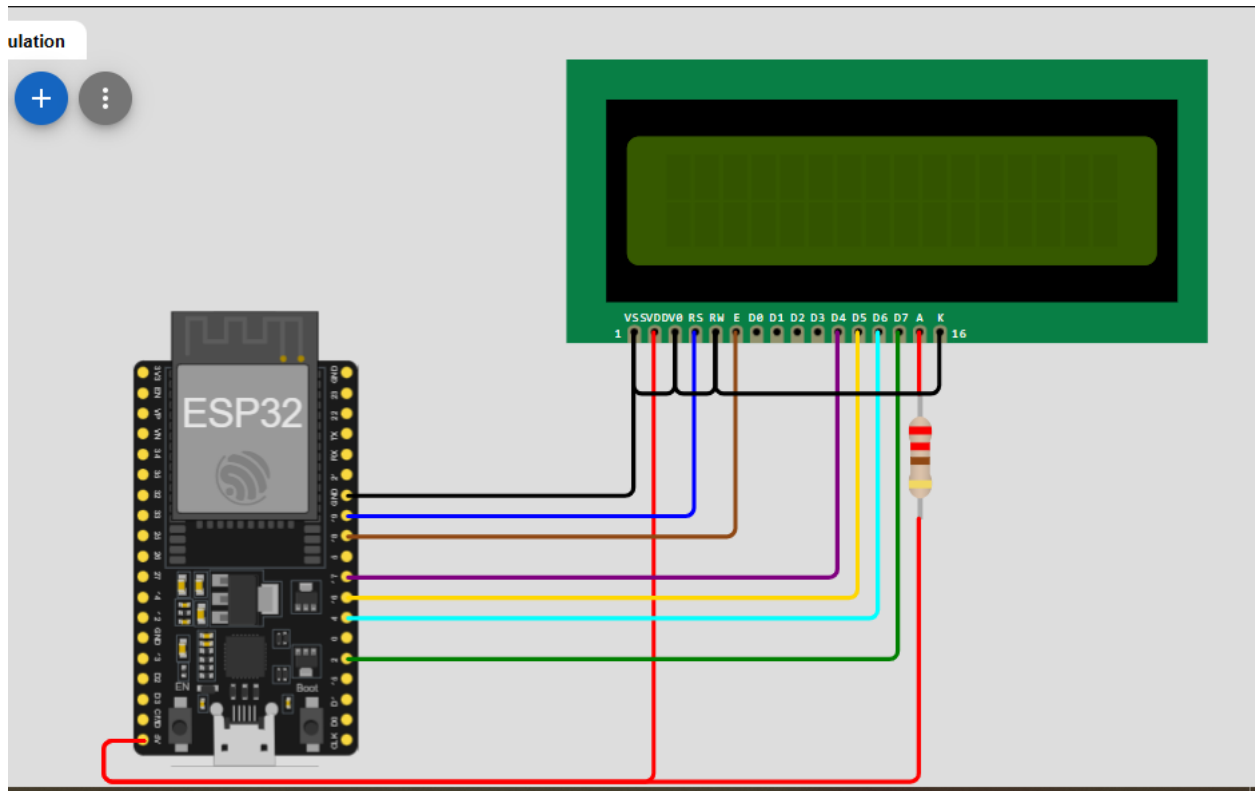


08.LCD 16x2 (Parallel)

- Write a program to:
 - Display your name and roll number



Pins of device	Pins of ESP32
VSS	GND
VDD	5V
V0	GND (via resistor)
RS	GPIO 19
RW	GND
E	GPIO 18
D0	-
D1	-
D2	-
D3	-
D4	GPIO 17
D5	GPIO 16
D6	GPIO 4
D7	GPIO 2
A	5V (via 220Ω)
K	GND

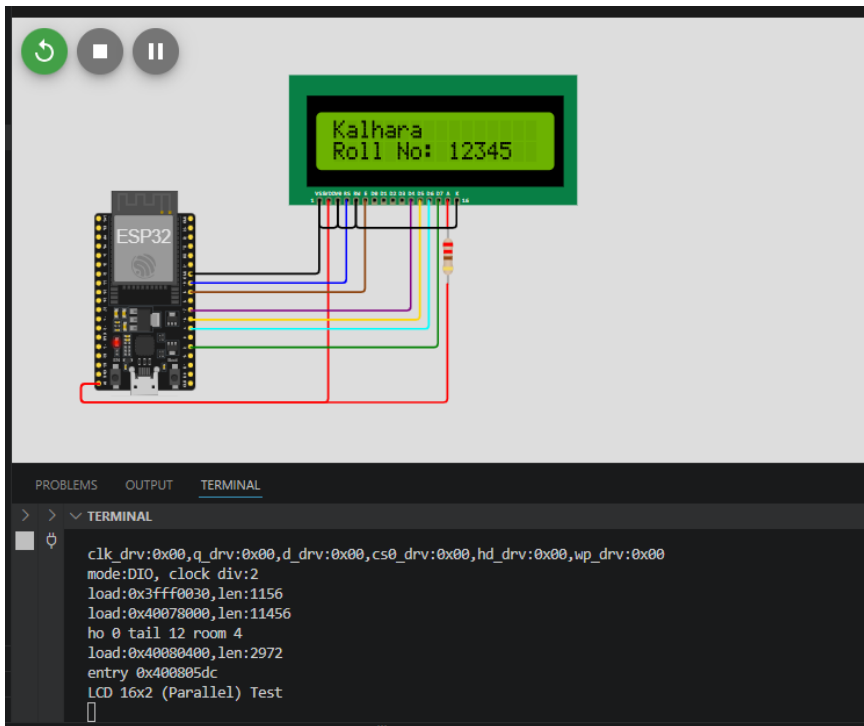


```
#include <Arduino.h>
#include <LiquidCrystal.h>
// Initialize the library with the numbers of the interface pins
// LiquidCrystal Lcd(RS, E, D4, D5, D6, D7)
const int rs = 19, en = 18, d4 = 17, d5 = 16, d6 = 4, d7 = 2;
LiquidCrystal Lcd(rs, en, d4, d5, d6, d7);

void setup() {
  // Set up the LCD's number of columns and rows:
  Lcd.begin(16, 2);
  Serial.begin(115200);
  Serial.println("LCD 16x2 (Parallel) Test");
  // Print your name on the first row (Row 0)
  Lcd.setCursor(0, 0);
  Lcd.print("Kalhara");

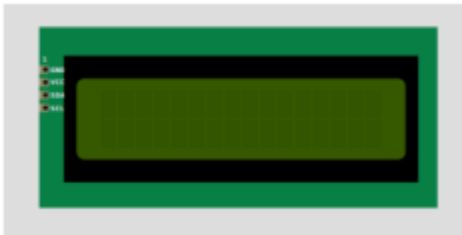
  // Move cursor to the second row (Row 1) and print roll number
  Lcd.setCursor(0, 1);
  Lcd.print("Roll No: 12345");
}

void loop() {
  // Nothing to repeat in the loop since the text is static
}
```

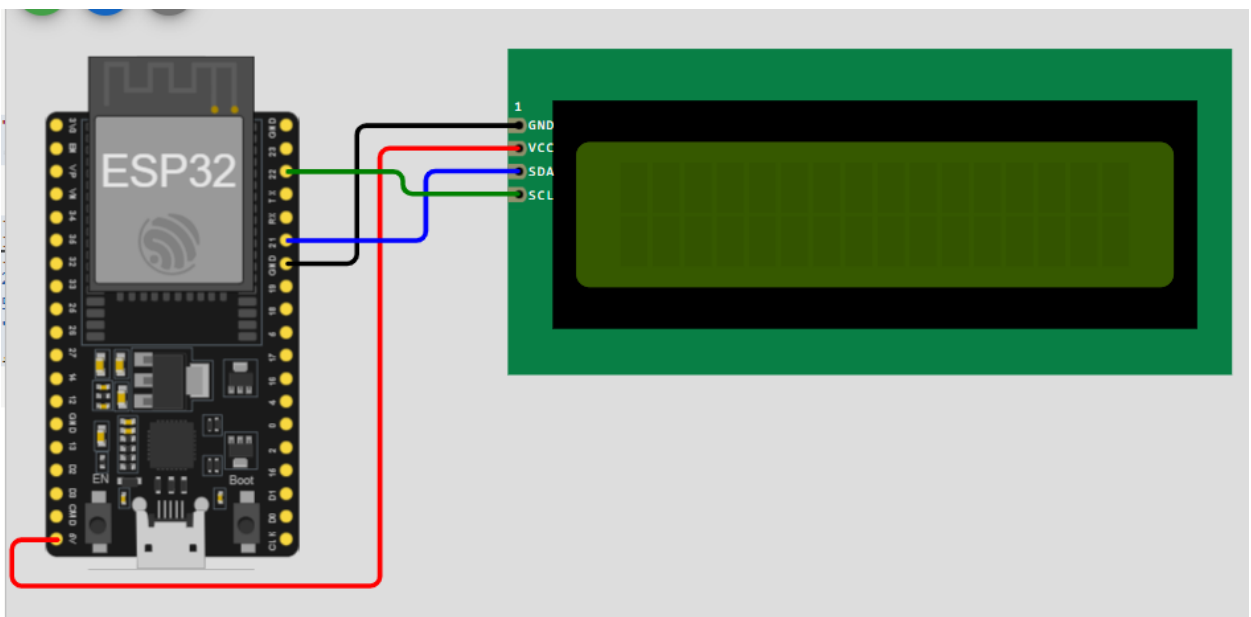
09.LCD 16x2 (I2C)

- Write a program to:
 - Display sensor values on LCD



Pins of device	Pins of ESP32
GND	GND
VCC	5V
SDA	GPIO 4
SCL	GPIO 5

21
11



```

#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Set the LCD address to 0x27 for a 16 chars and 2 line display
// (If 0x27 doesn't work, your module might use 0x3F)
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  Serial.begin(115200);
  Serial.println("LCD 16x2 (I2C) Test");
  // Initialize the LCD connected via I2C
  lcd.init();
  // Turn on the backlight
  lcd.backlight();
  // Print a static header on the first row
  lcd.setCursor(0, 0);
  lcd.print("Sensor Monitor");
}

void loop() {
  // Simulate reading a sensor value (e.g., generating a value between 0 and 100)
  int mockSensorValue = analogRead(34) / 41; // Scaled down for example display

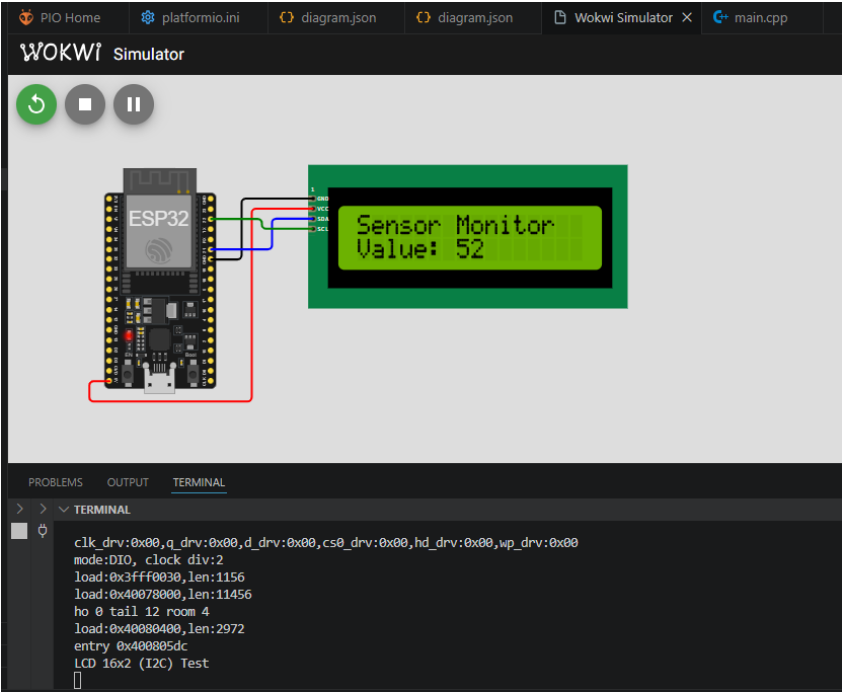
  // Move cursor to the second row (Column 0, Row 1)
  lcd.setCursor(0, 1);
  lcd.print("Value: ");

  // Print the dynamic sensor value
  lcd.print(mockSensorValue);

  // Clear any leftover trailing digits if the value drops (e.g., from 100 to 9)
  lcd.print("    ");

  // Update every 500 milliseconds
  delay(500);
}

```

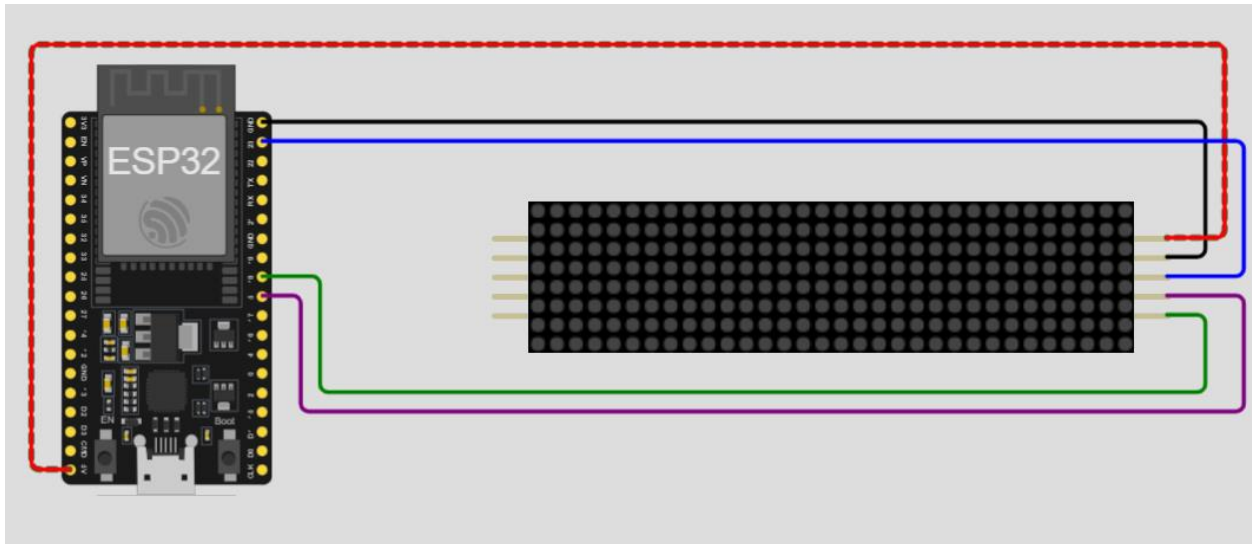


10.MAX7219 LED Dot Matrix

- Write a program to:
 - Display scrolling text



Pins of device	Pins of ESP32
V+	5V
GND	GND
DIN	GPIO 23
CS	GPIO 5
CLK	GPIO 18
V+.2	-
GND.2	-
DOUT	-
CS.2	-
CLK.2	-



```
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

// Try PAROLA_HW instead - often fixes display corruption
#define HARDWARE_TYPE MD_MAX72XX::PAROLA_HW
#define MAX_DEVICES 4

const int DATA_PIN = 23; // DIN
const int CS_PIN = 5; // CS
const int CLK_PIN = 18; // CLK

MD_Parola myDisplay = MD_Parola(HARDWARE_TYPE, DATA_PIN, CLK_PIN, CS_PIN,
MAX_DEVICES);

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\nMAX7219 LED Matrix - PAROLA_HW Test");

  myDisplay.begin();

  // Reverse all devices to fix byte order
  for (uint8_t i = 0; i < MAX_DEVICES; i++) {
    myDisplay.setFont(i, NULL);
  }

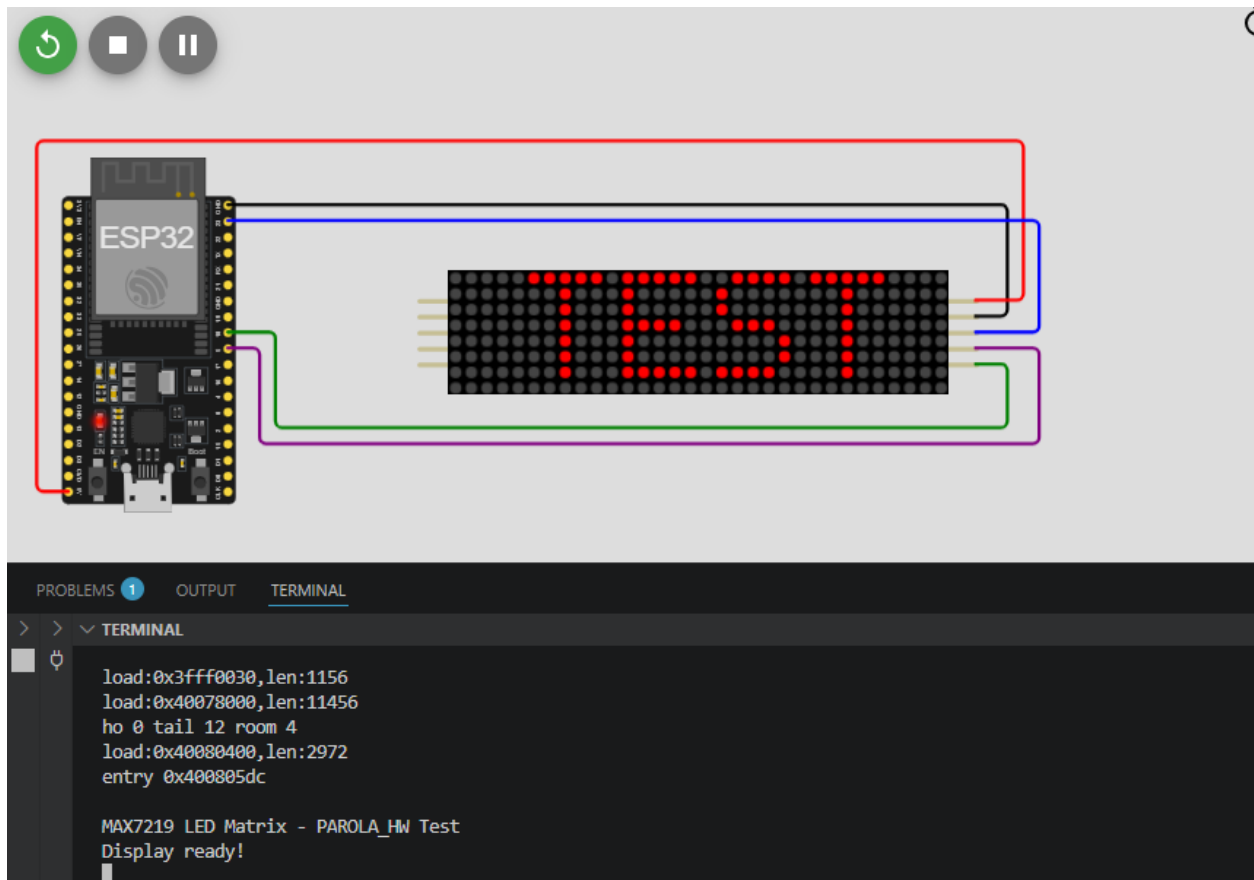
  myDisplay.setIntensity(10);
  myDisplay.displayClear();
  delay(500);
```

```

    Serial.println("Display ready!");
    myDisplay.displayText("TEST", PA_CENTER, 50, 1000, PA_PRINT, PA_PRINT);
}

void loop() {
    if (myDisplay.displayAnimate()) {
        myDisplay.displayReset();
    }
}

```

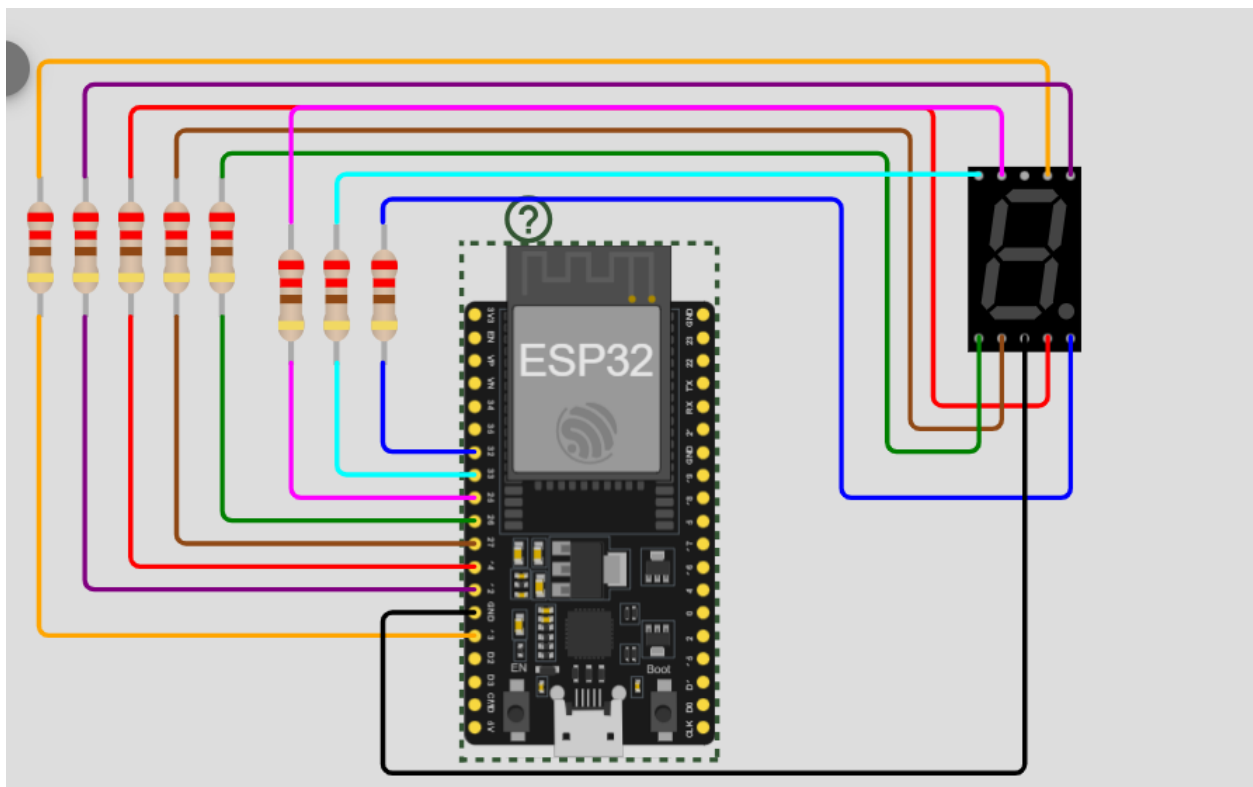
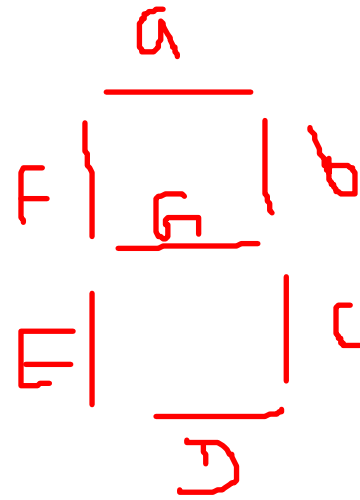


11.Seven Segment Display

- Write a program to:
 - Display digits 0–9 sequentially



Pins of device	Pins of ESP32
A	GPIO 13 (via 220Ω)
B	GPIO 12 (via 220Ω)
C	GPIO 14 (via 220Ω)
D	GPIO 27 (via 220Ω)
E	GPIO 26 (via 220Ω)
F	GPIO 25 (via 220Ω)
G	GPIO 33 (via 220Ω)
DP	GPIO 32 (via 220Ω)
COM.1	5V
COM.2	-



```
#include <Arduino.h>
// Define segment pins according to your image table
const int segA = 13;
const int segB = 12;
const int segC = 14;
const int segD = 27;
const int segE = 26;
const int segF = 25;
const int segG = 33;
```

```

const int segDP = 32; // Decimal Point pin

// Group all segment pins into an array for easy initialization
const int segments[] = {segA, segB, segC, segD, segE, segF, segG, segDP};

// Byte patterns for digits 0-9 (Common Cathode: 1 = ON, 0 = OFF)
// Bits correspond to: {A, B, C, D, E, F, G}
const byte digitPatterns[10][7] = {
    {1, 1, 1, 1, 1, 1, 0}, // 0
    {0, 1, 1, 0, 0, 0, 0}, // 1
    {1, 1, 0, 1, 1, 0, 1}, // 2
    {1, 1, 1, 1, 0, 0, 1}, // 3
    {0, 1, 1, 0, 0, 1, 1}, // 4
    {1, 0, 1, 1, 0, 1, 1}, // 5
    {1, 0, 1, 1, 1, 1, 1}, // 6
    {1, 1, 1, 0, 0, 0, 0}, // 7
    {1, 1, 1, 1, 1, 1, 1}, // 8
    {1, 1, 1, 1, 0, 1, 1}  // 9
};

// Function prototype
void displayDigit(int num);

void setup() {
    Serial.begin(115200);
    Serial.println("Seven Segment Display Test");
    // Set all segment pins as OUTPUT
    for (int i = 0; i < 8; i++) {
        pinMode(segments[i], OUTPUT);
        digitalWrite(segments[i], LOW); // Turn all segments OFF initially
    }
}

void loop() {
    // Loop through digits 0 to 9
    for (int digit = 0; digit < 10; digit++) {
        displayDigit(digit);
        delay(1000); // Wait 1 second before displaying the next number
    }
}

// Helper function to update the physical pins with a digit pattern
void displayDigit(int num) {
    for (int seg = 0; seg < 7; seg++) {
        digitalWrite(segments[seg], digitPatterns[num][seg]);
    }
}

```

```
}  
// Keep the decimal point (DP) turned off  
digitalWrite(segDP, LOW);  
}
```

